

CS61C: Great Ideas in Computer Architecture (aka Machine Structures)

Lecture 19: Pipelined Datapath II

Instructor: Justin Yokota

Agenda

- Pipelined RISC-V Datapath
- Hazards Introduction
- Structural Hazards
- Data Hazards
 - What are Data Hazards
 - Forwarding
 - Loads
- Control Hazards
- Superscalar Processors and Calculating CPI

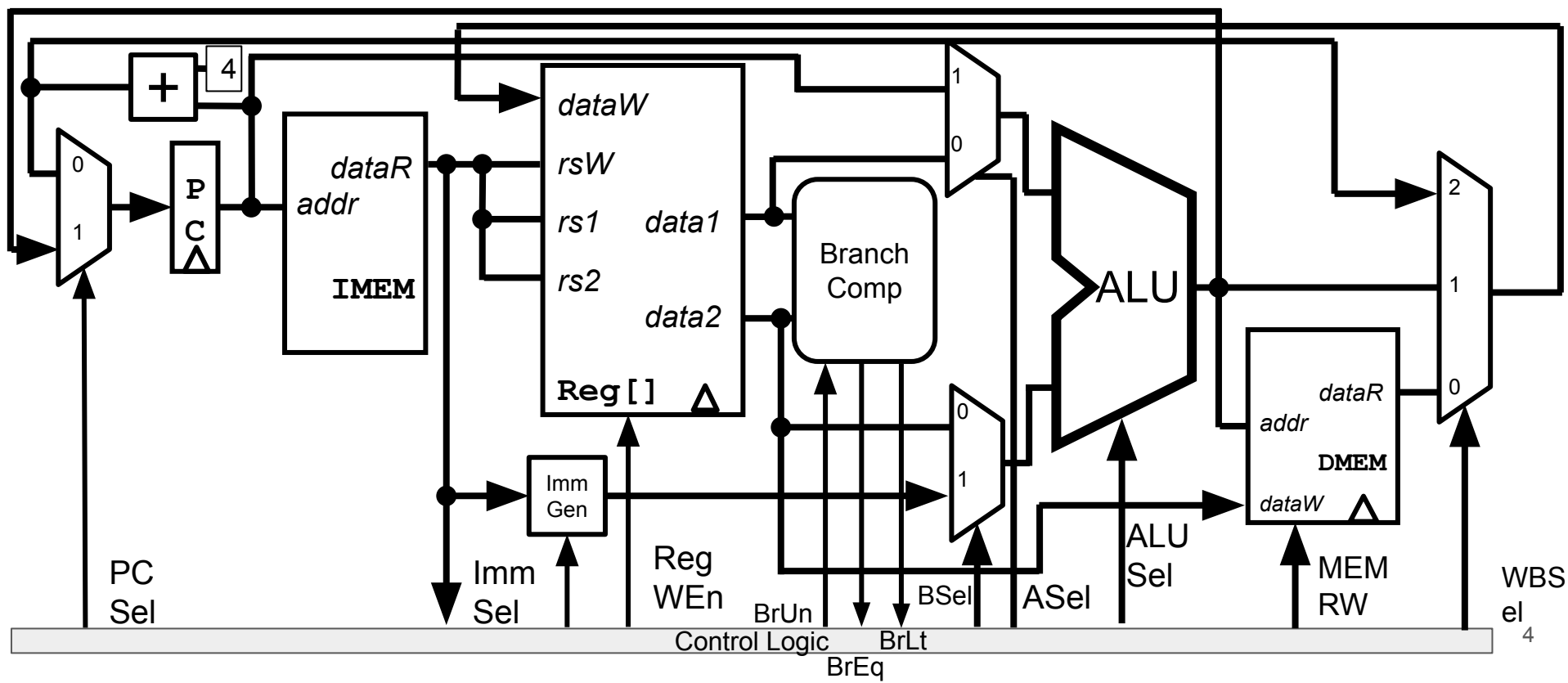
Agenda

- **Pipelined RISC-V Datapath**
- Hazards Introduction
- Structural Hazards
- Data Hazards
 - What are Data Hazards
 - Forwarding
 - Loads
- Control Hazards
- Superscalar Processors and Calculating CPI

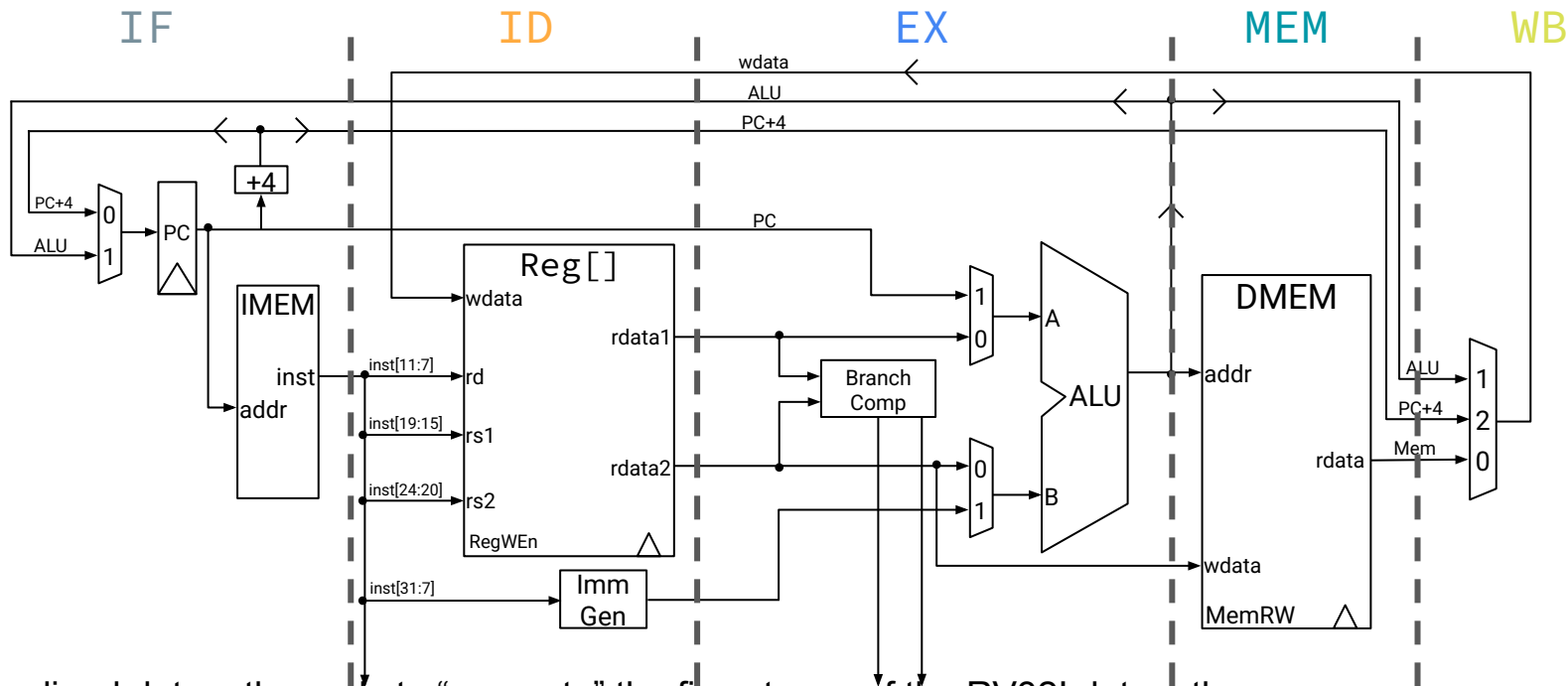
Complete RISC-V Datapath!

CS 61C

Summer 2026



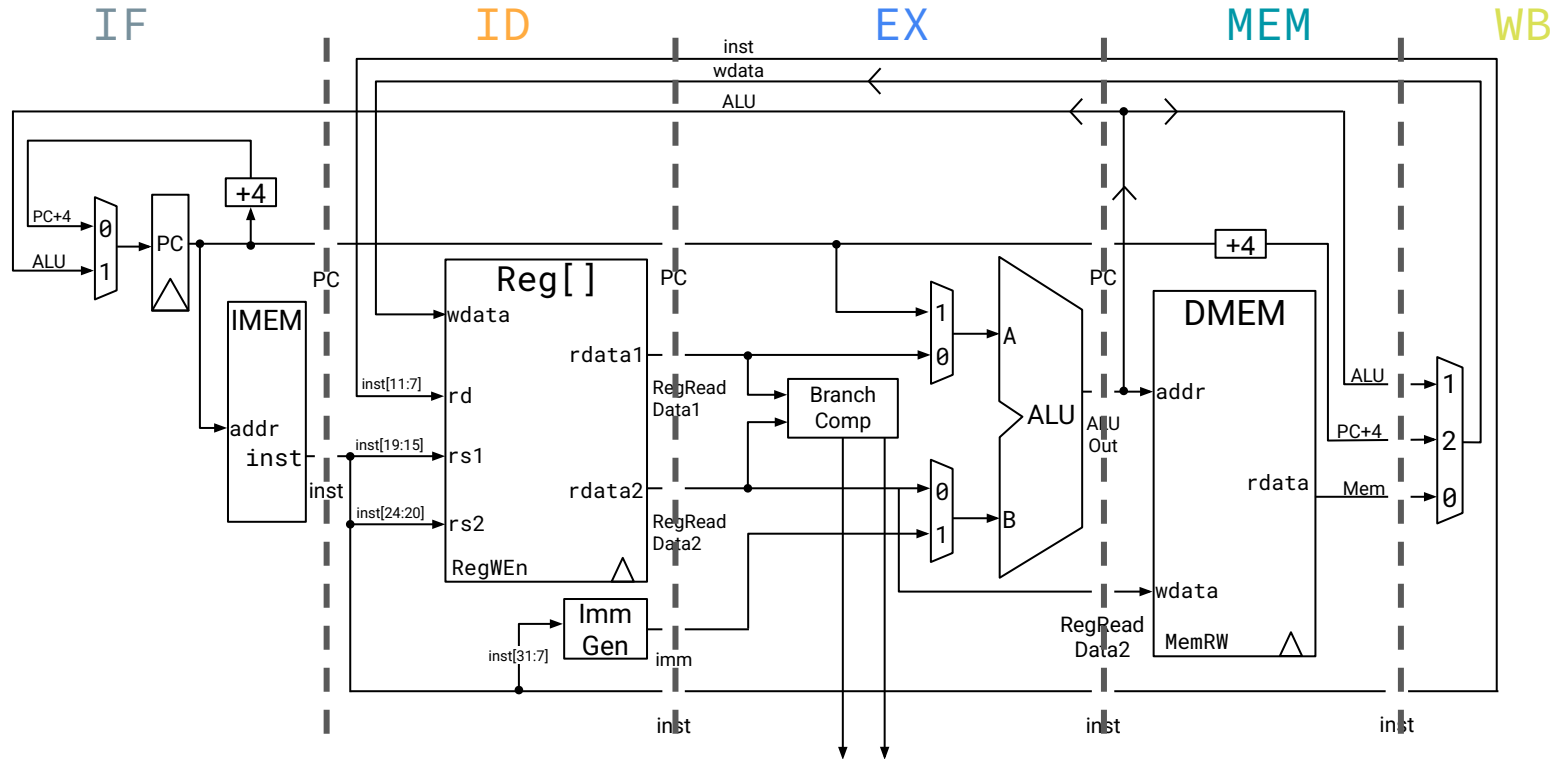
5-Stage Datapath for RV32I



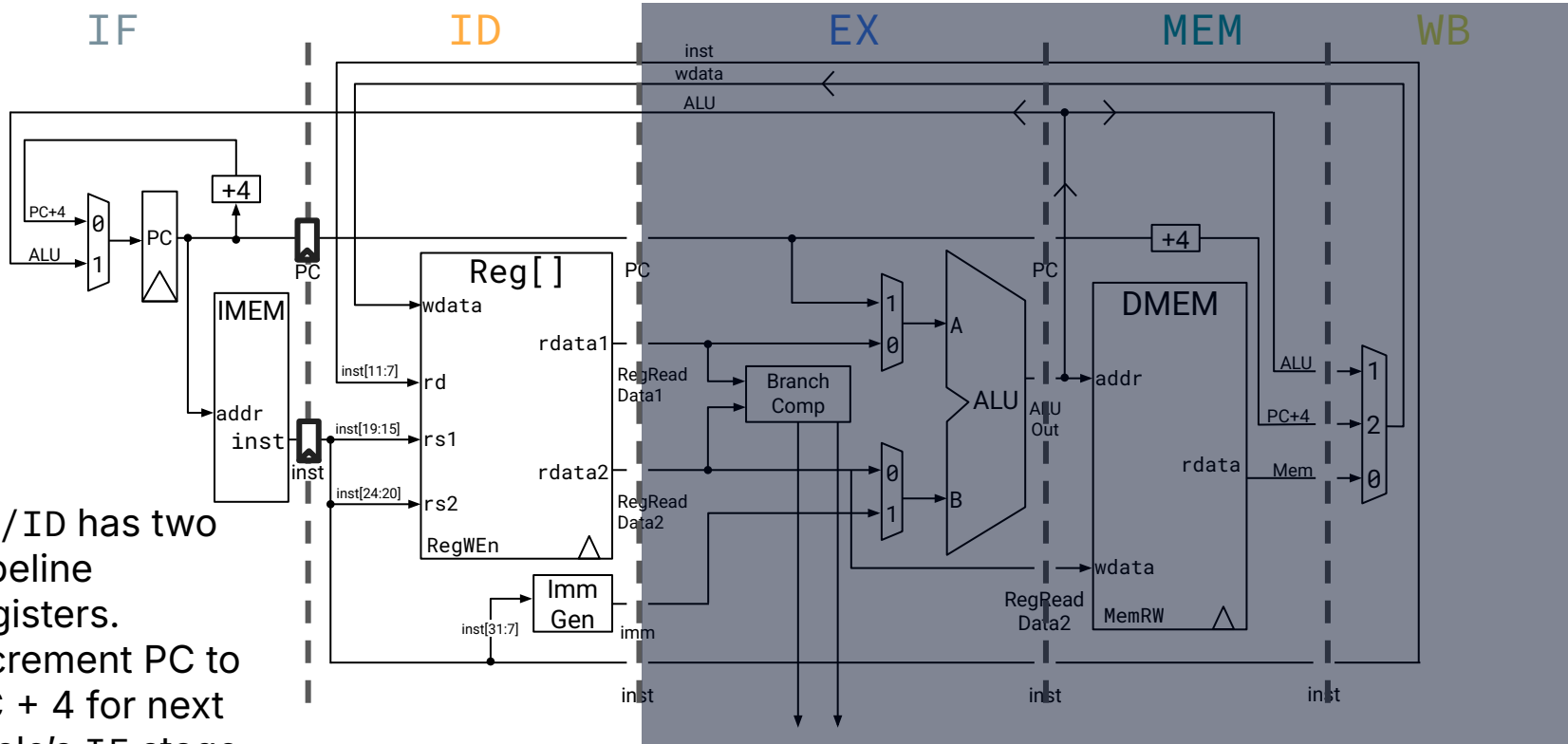
A pipelined datapath needs to “separate” the five stages of the RV32I datapath.

- Each stage needs to process data from a different instruction.

5-Stage Datapath for RV32I

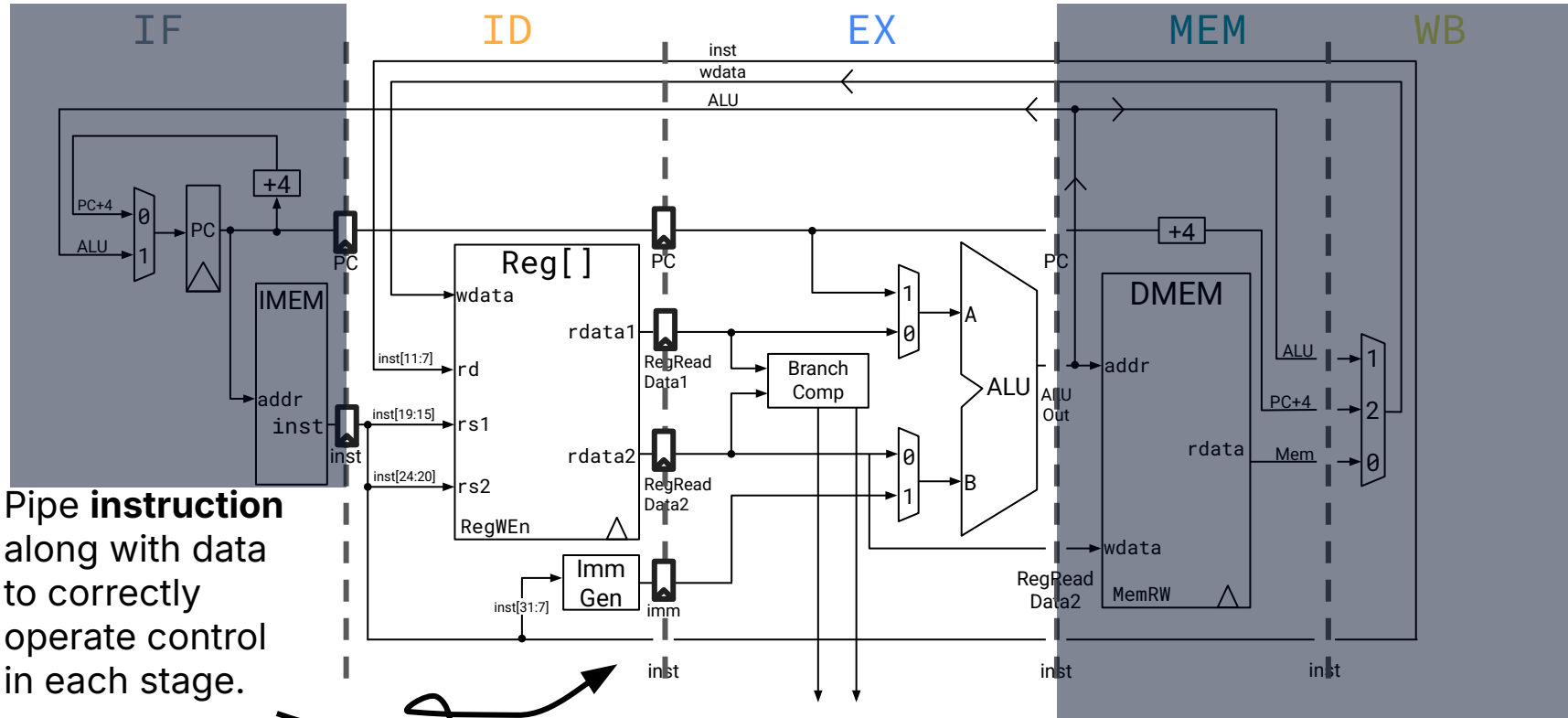


Datapath Registers: IF/ID



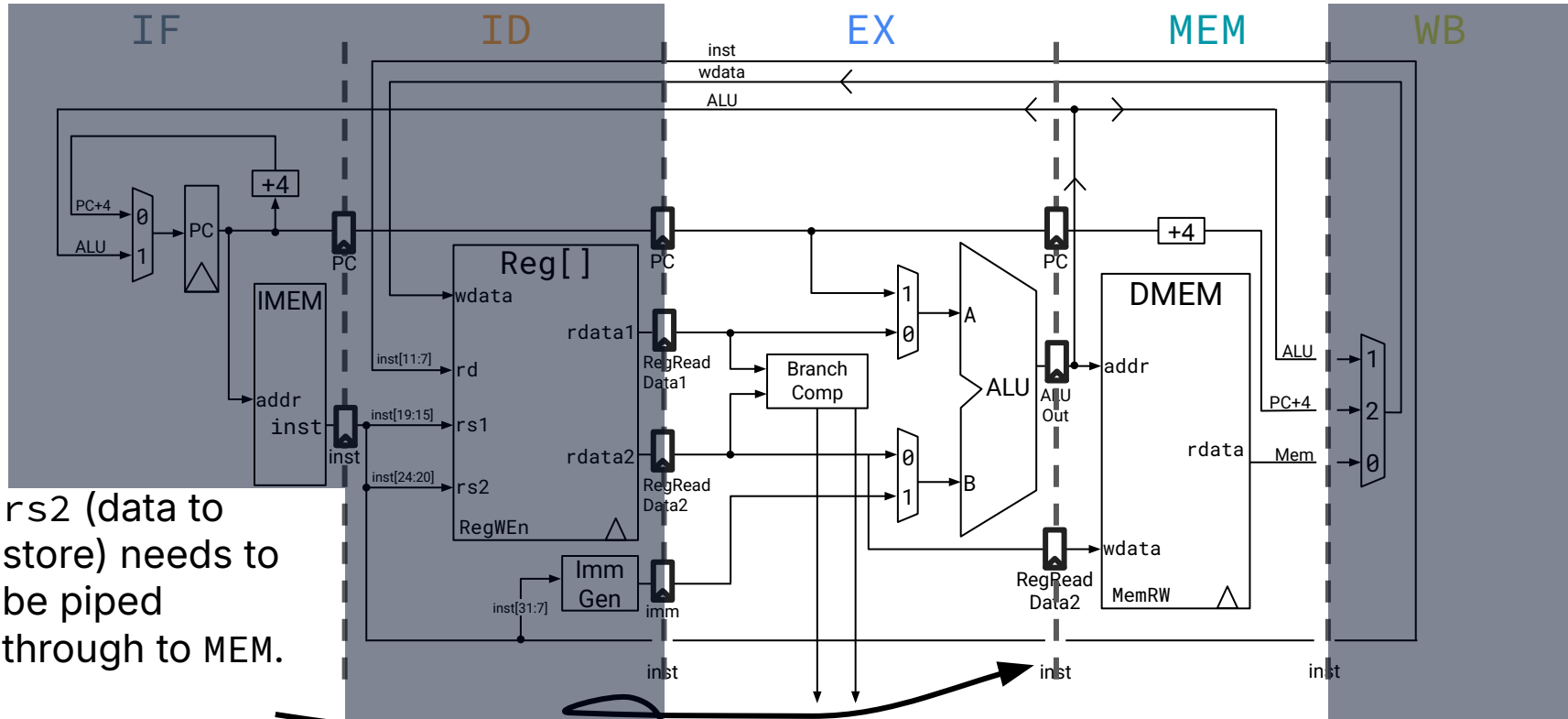
- IF/ID has two pipeline registers.
- Increment PC to $PC + 4$ for next cycle's IF stage.

Datapath Registers: ID/EX

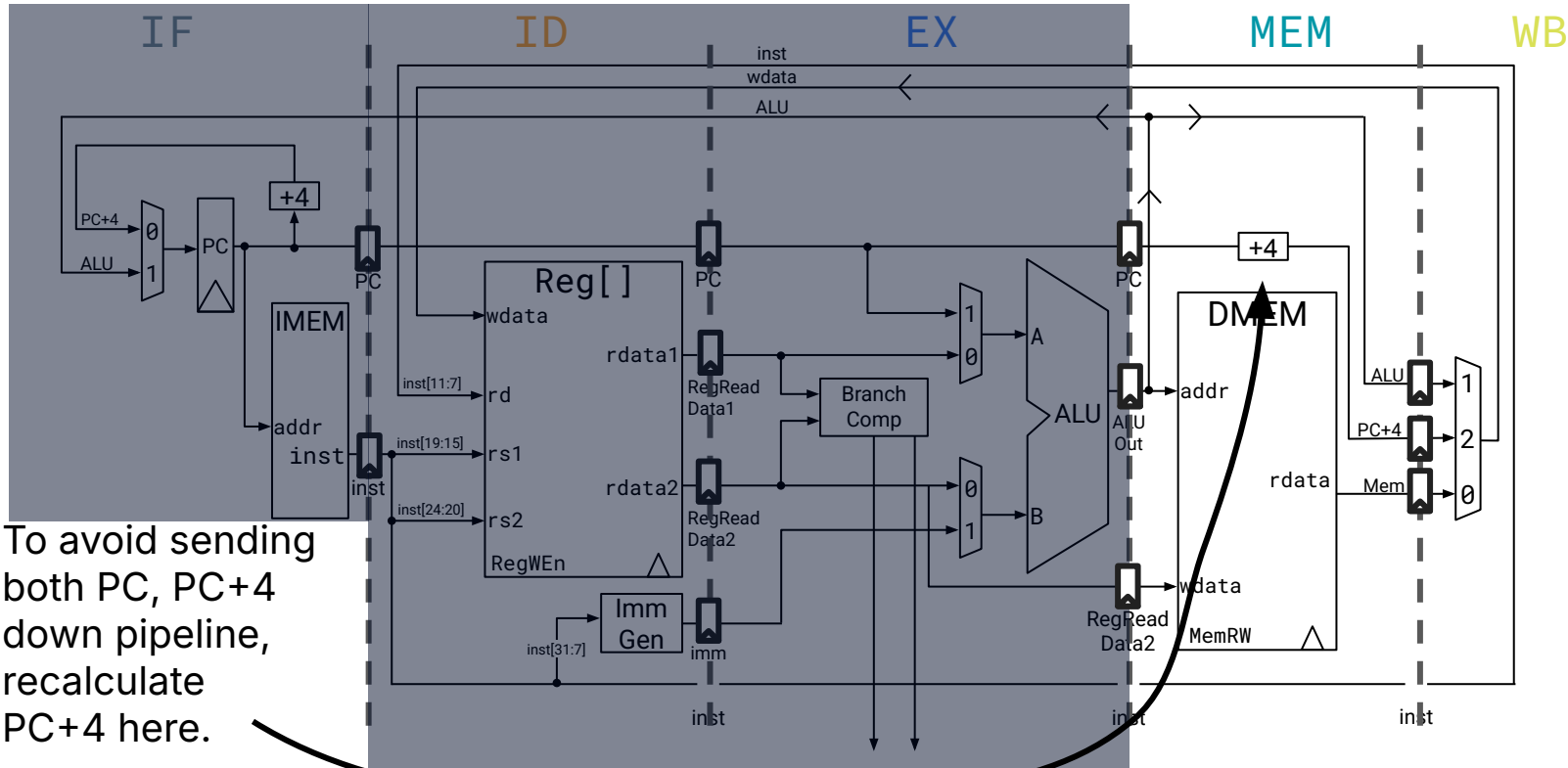


- Pipe **instruction** along with data to correctly operate control in each stage.

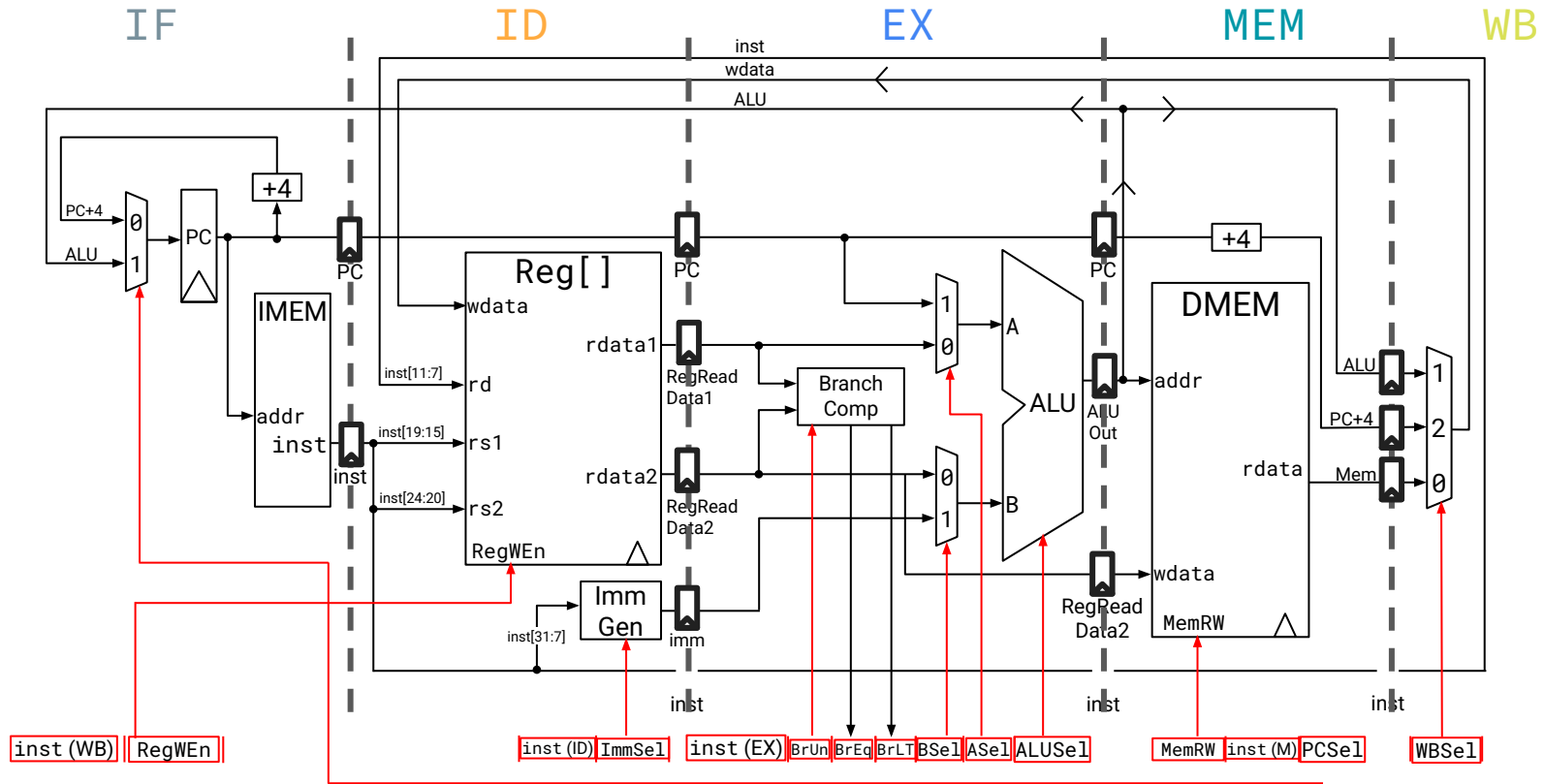
Datapath Registers: EX/MEM



Datapath Registers: MEM/WB

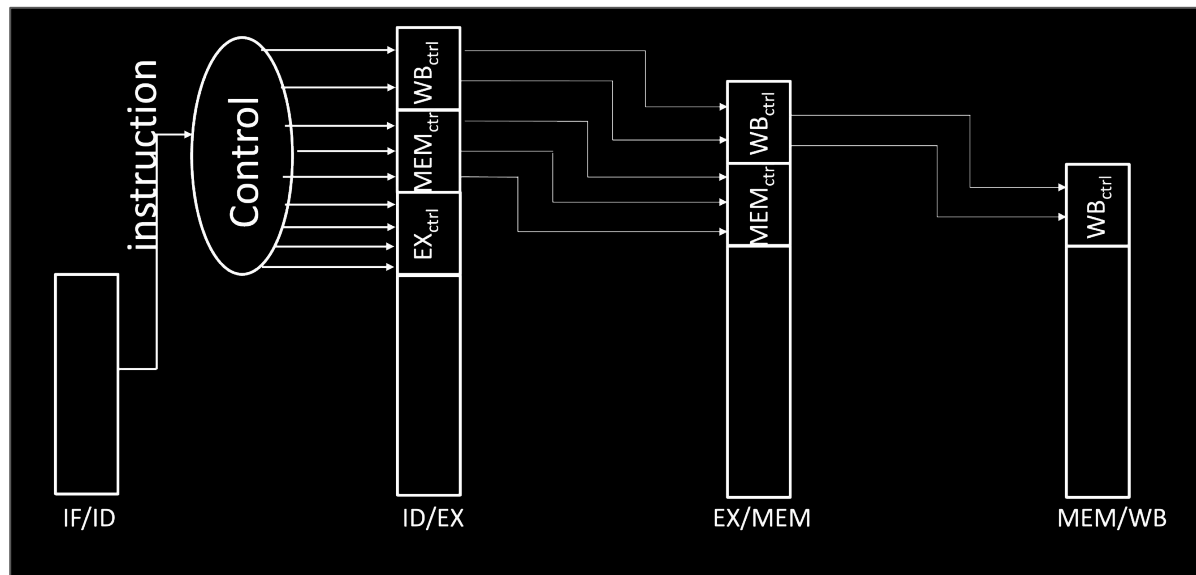


Control Is Also Pipelined



Control Is Also Pipelined

- Control signals are derived from the instruction.
 - Control information for later stages is also stored in **pipeline registers**.
- One strategy:
 - Compute as many control signals as possible during instruction decode (**ID**).
 - Then, pipeline control “words” between stages.



Pipelining and ISA Design

The RISC-V ISA is designed for **pipelining**:

- All instructions are 32 bits wide.
 - Easy to fetch (IF) and decode (ID), each in one clock cycle.
 - Contrast with CISC (Complex) x86: 1- to 15-byte-wide instructions!
- A small set of standard instruction formats
 - Can decode/read registers in one stage (2nd, ID).
- Load/store addressing conceptually:
 - Calculate address in 3rd stage (EX) with ALU; and
- Access memory in one stage (4th, MEM).
 - Memory operands are all aligned
 - Memory access takes only one cycle.

The **5-stage pipeline** we have studied is commonplace in many devices: cars, appliances, etc.

Agenda

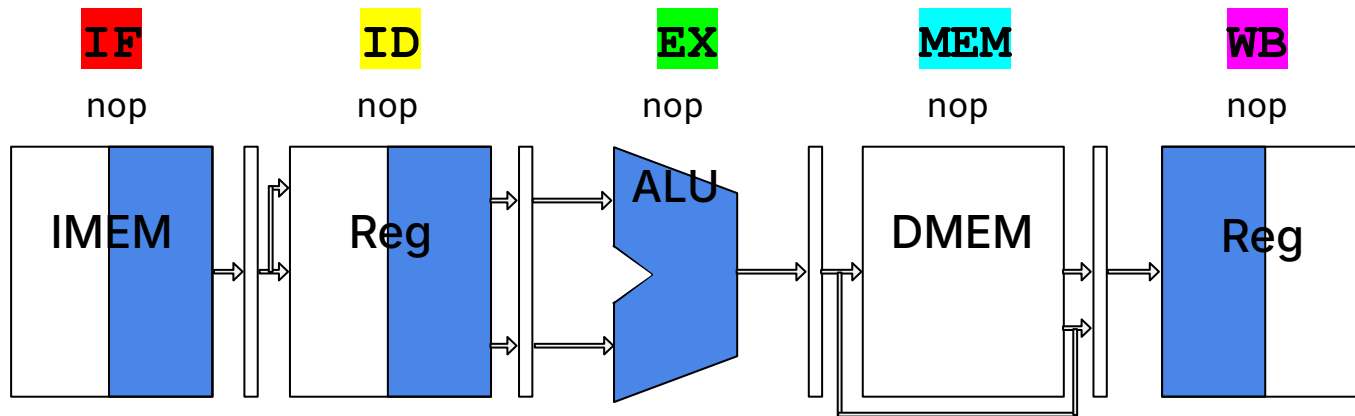
- Pipelined RISC-V Datapath
- **Hazards Introduction**
- Structural Hazards
- Data Hazards
 - What are Data Hazards
 - Forwarding
 - Loads
- Control Hazards
- Superscalar Processors and Calculating CPI

Pipelining

Let's try running some code on this pipeline

When starting a program, the pipeline should be filled with instructions that do nothing:

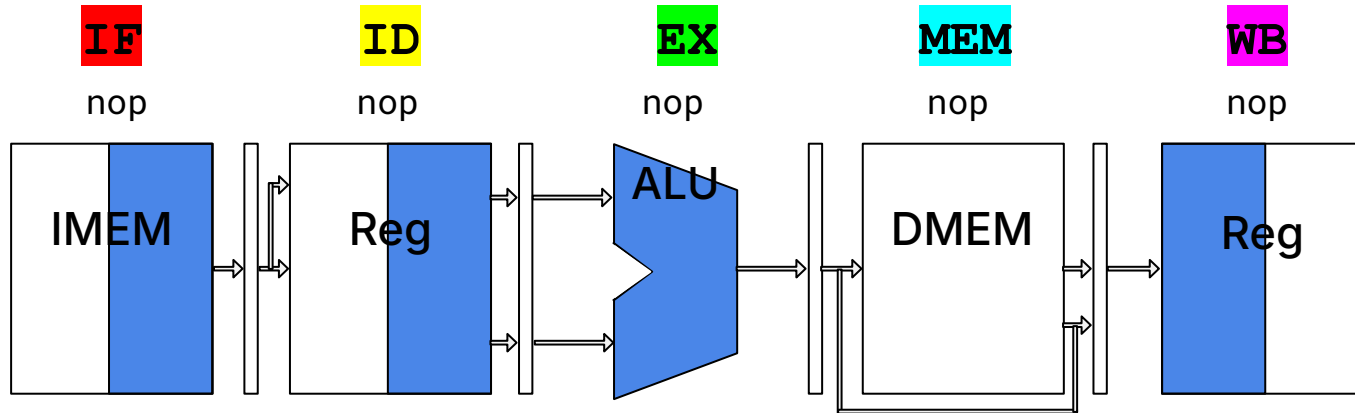
`nop` = NO oPeration (or No OPeration)



Pipelining

```
addi t0 x0 50  
addi t1 x0 100  
addi t2 x0 150
```

PC	0
t0	0
t1	0
t2	0



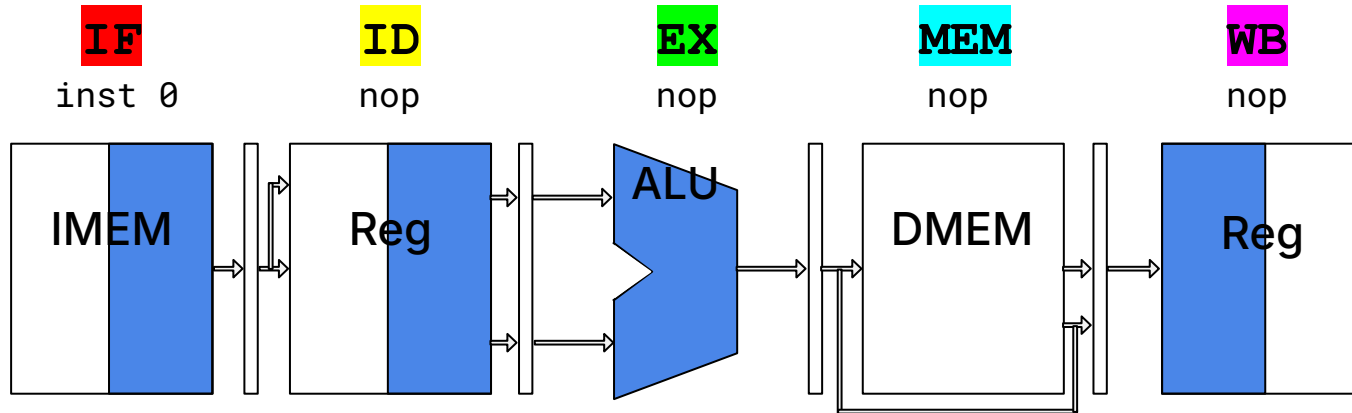
Pipelining: Cycle 0

```
addi t0 x0 50
```

```
addi t1 x0 100
```

```
addi t2 x0 150
```

PC	0
t0	0
t1	0
t2	0



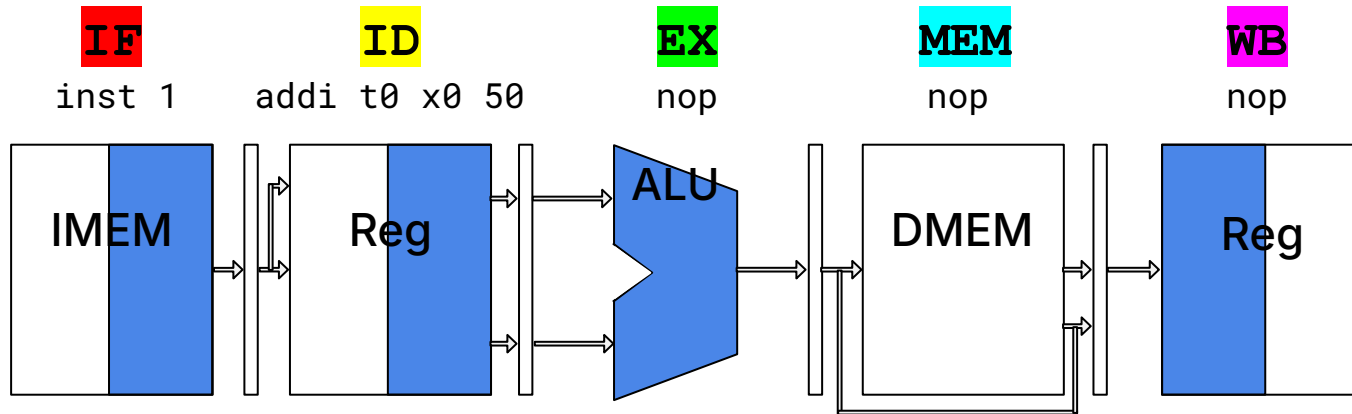
Pipelining: Cycle 1

addi t0 x0 50

addi t1 x0 100

addi t2 x0 150

PC	4
t0	0
t1	0
t2	0



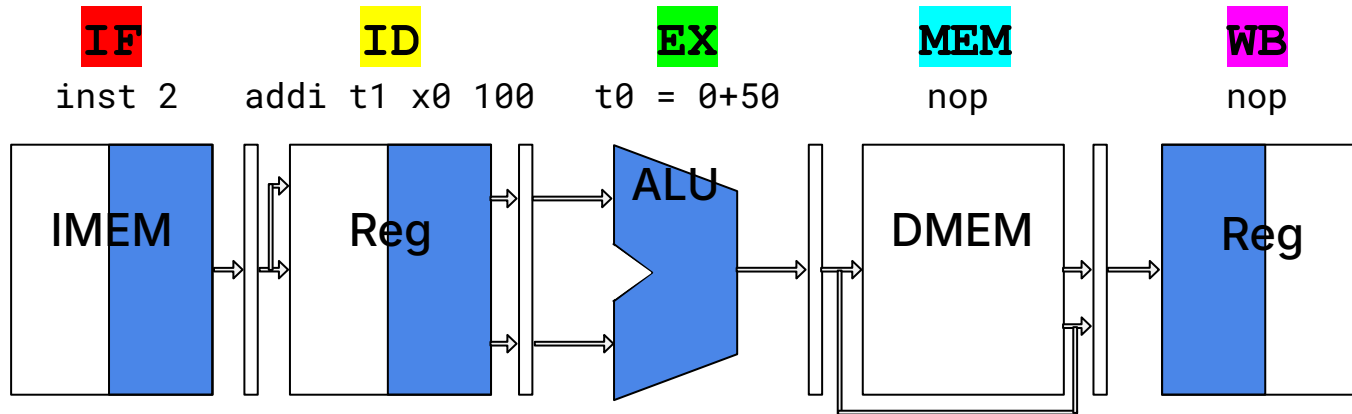
Pipelining: Cycle 2

addi t0 x0 50

addi t1 x0 100

addi t2 x0 150

PC	8
t0	0
t1	0
t2	0



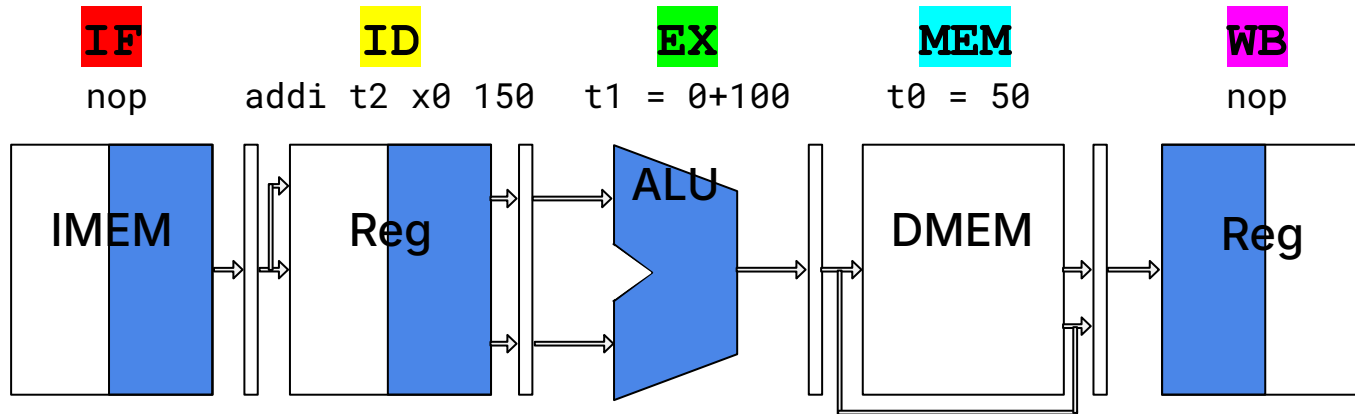
Pipelining: Cycle 3

```
addi t0 x0 50
```

```
addi t1 x0 100
```

```
addi t2 x0 150
```

PC	12
t0	0
t1	0
t2	0



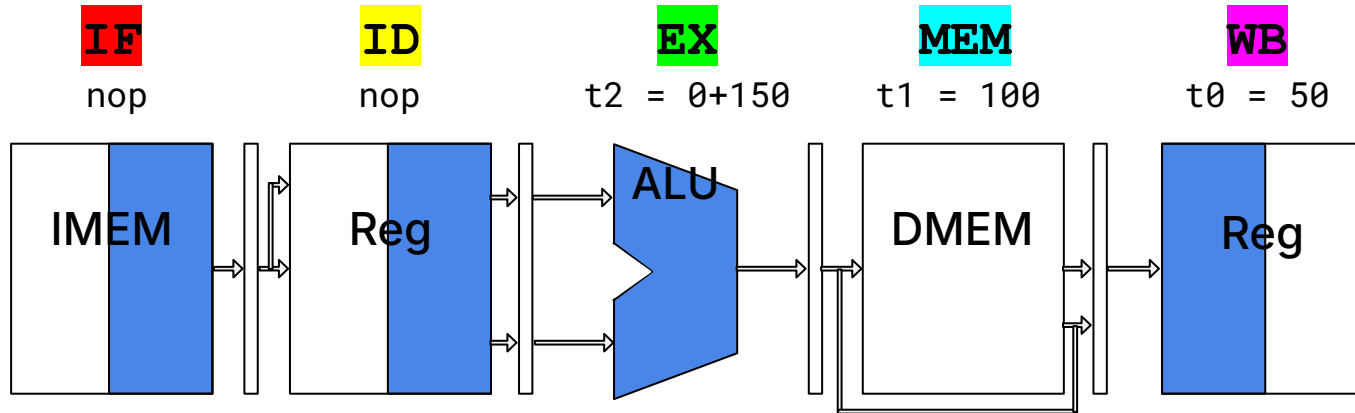
Pipelining: Cycle 4

addi t0 x0 50

addi t1 x0 100

addi t2 x0 150

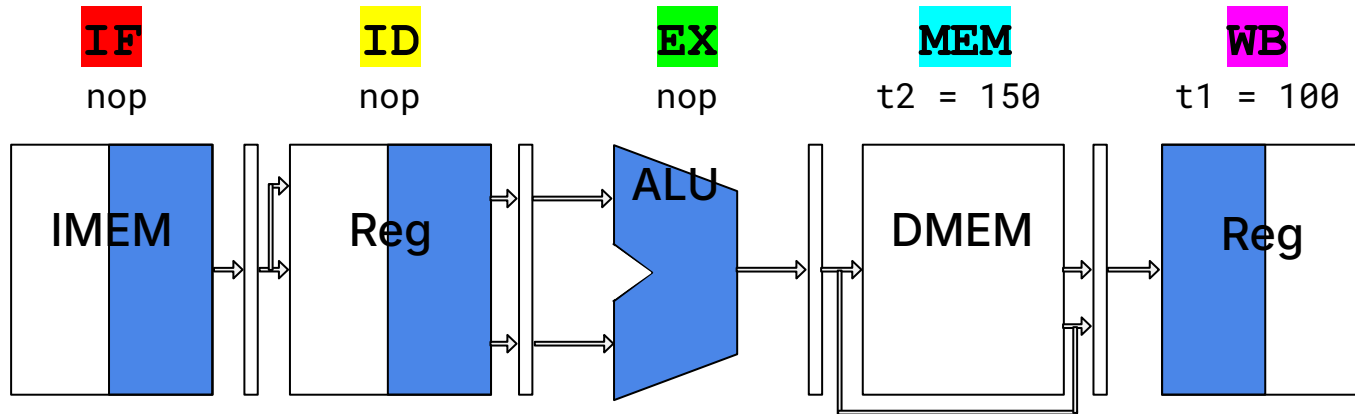
PC	16
t0	0
t1	0
t2	0



Pipelining: Cycle 5

```
addi t0 x0 50  
addi t1 x0 100  
addi t2 x0 150
```

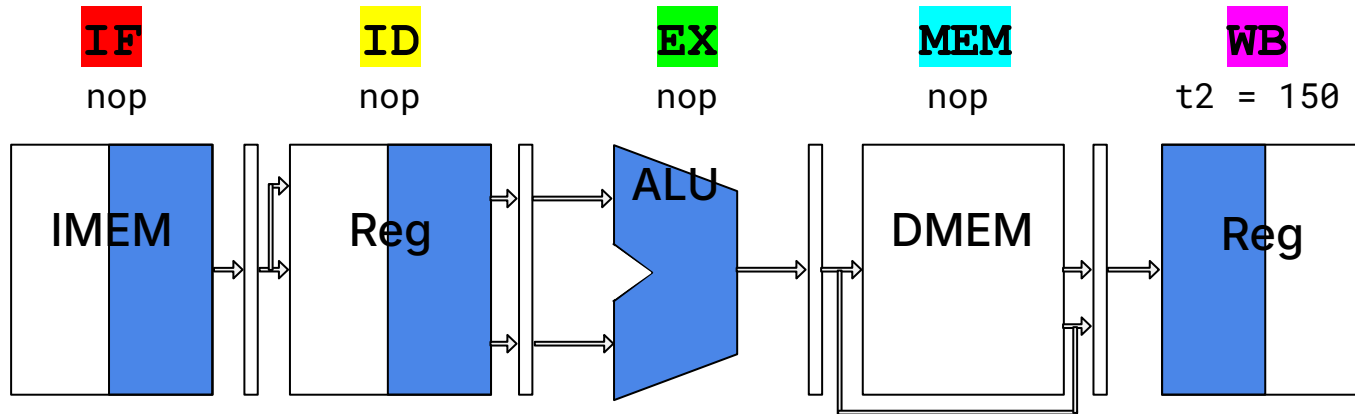
PC	20
t0	50
t1	0
t2	0



Pipelining: Cycle 6

```
addi t0 x0 50  
addi t1 x0 100  
addi t2 x0 150
```

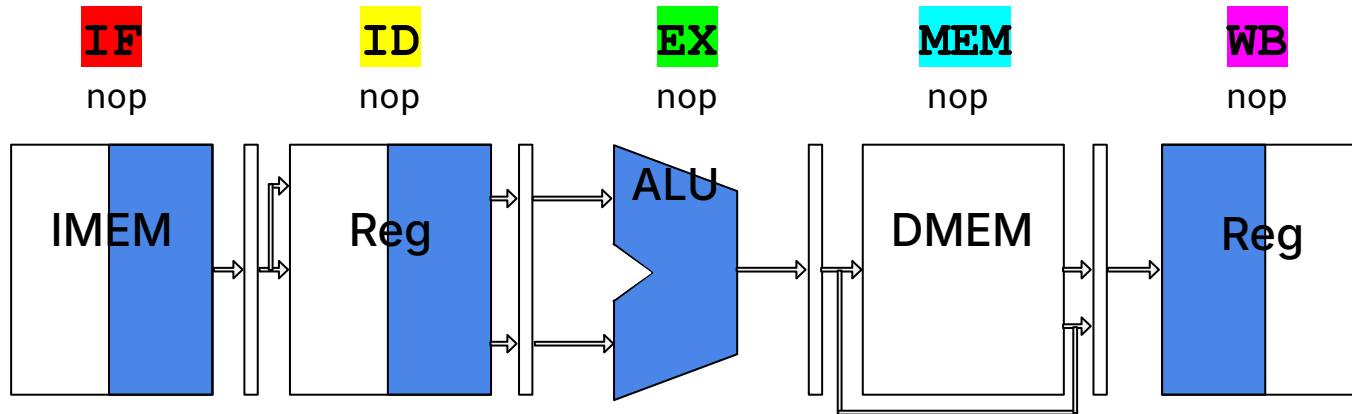
PC	24
t0	50
t1	100
t2	0



Pipelining: Cycle 7

```
addi t0 x0 50  
addi t1 x0 100  
addi t2 x0 150
```

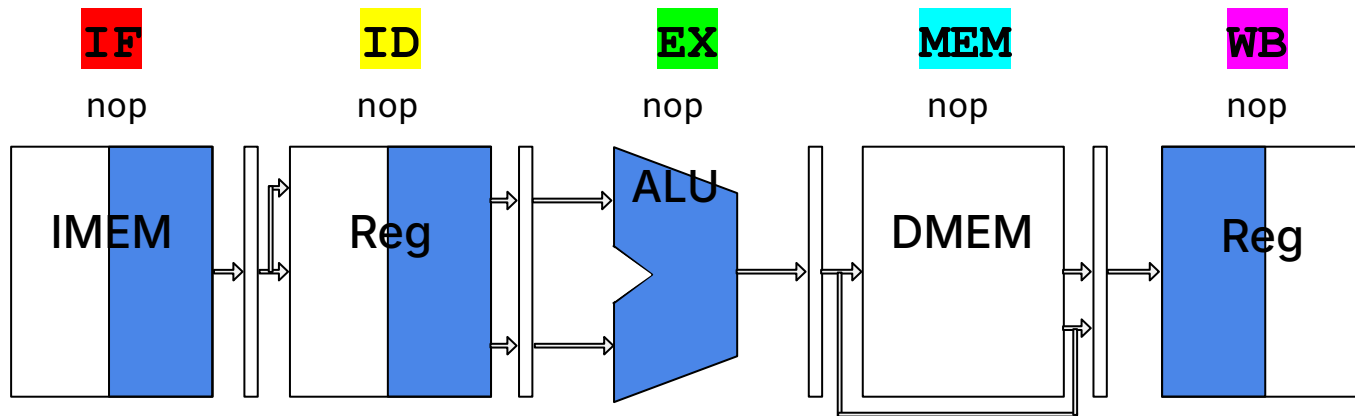
PC	28
t0	50
t1	100
t2	150



Pipelining Example

This code works! But... not all code will work.

When a naive pipeline causes errors, this is called a **hazard**. Three main classes of hazards.



Three Types of Pipeline Hazards

A **hazard** is a situation in which a planned instruction cannot execute in the “proper” clock cycle.

1. Structural hazard

- a. Hardware does not support access across multiple instructions in the same cycle.

2. Data hazard

- a. Instructions have data dependency.
- b. Need to wait for previous instruction to complete its data read/write.

3. Control hazard

- a. Flow of execution depends on previous instruction.

Stalling

- **Stalling** is one inefficient solution to resolving any type of hazard:
 - If we add 4 stalls between each instruction, we get the same as the single-stage pipeline
 - Some instructions stall until resources are “ready.”
 - Requires **extra pipeline state** to prevent unintended state changes in stalled stages (e.g., register/memory writes).
- With stalling, instructions get delayed, and performance suffers.
 - We’d like to **avoid stalling** instructions where possible!

```
add s0 t0 t1
sub t2 s0 t0
or  t6 s0 t3
```



```
add s0 t0 t1
nop
...
nop
sub t2 s0 t0
or  t6 s0 t3
```

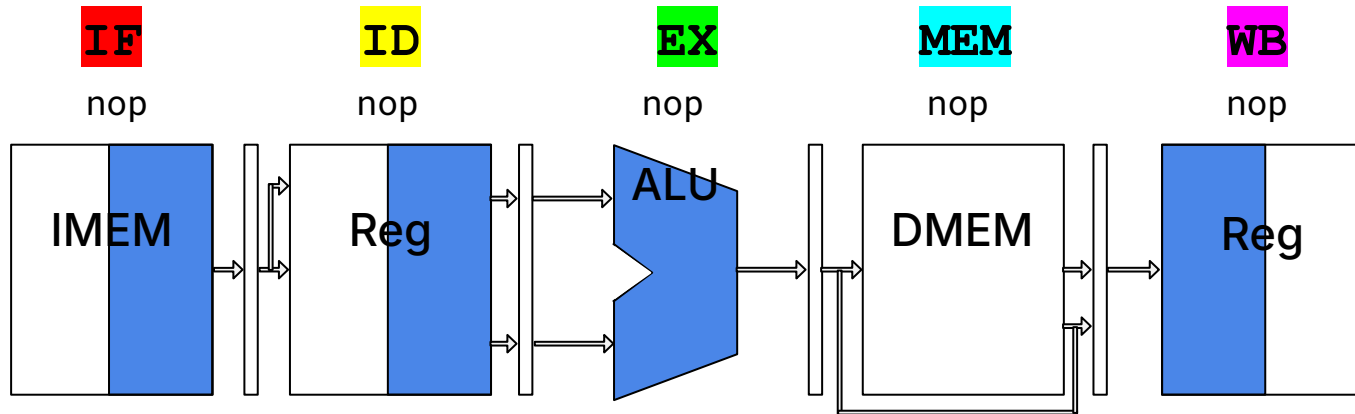
Agenda

- Pipelined RISC-V Datapath
- Hazards Introduction
- **Structural Hazards**
- Data Hazards
 - What are Data Hazards
 - Forwarding
 - Loads
- Control Hazards
- Superscalar Processors and Calculating CPI

Structural Hazard

```
addi t0 x0 50  
sw t2 16(x0)  
nop  
addi t0 x0 11  
nop
```

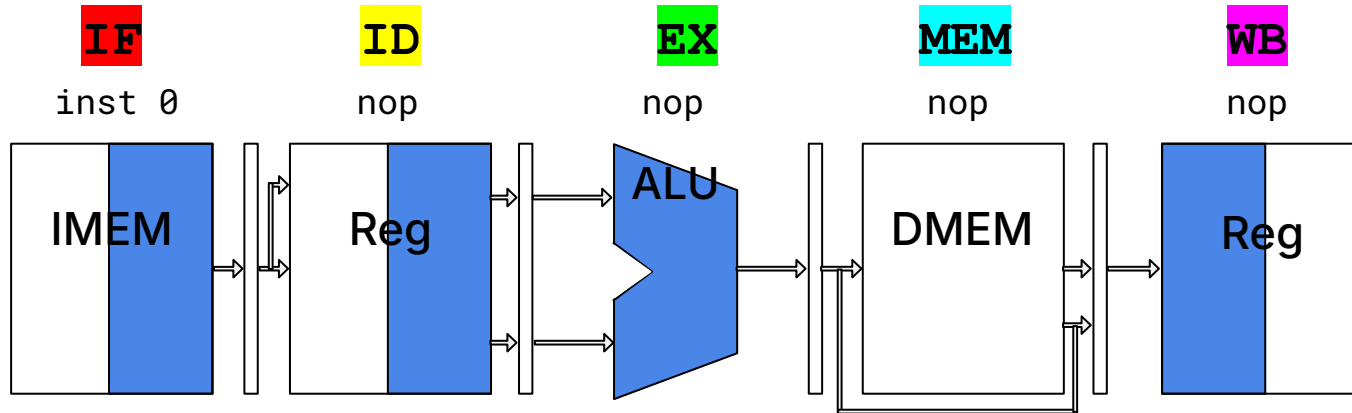
PC	0
t0	0
t1	0
t2	0x0012F313



Structural Hazard: Cycle 0

```
addi t0 x0 50  
sw t2 16(x0)  
nop  
addi t0 x0 11  
nop
```

PC	0
t0	0
t1	0
t2	0x0012F313



Structural Hazard: Cycle 1

```
addi t0 x0 50
```

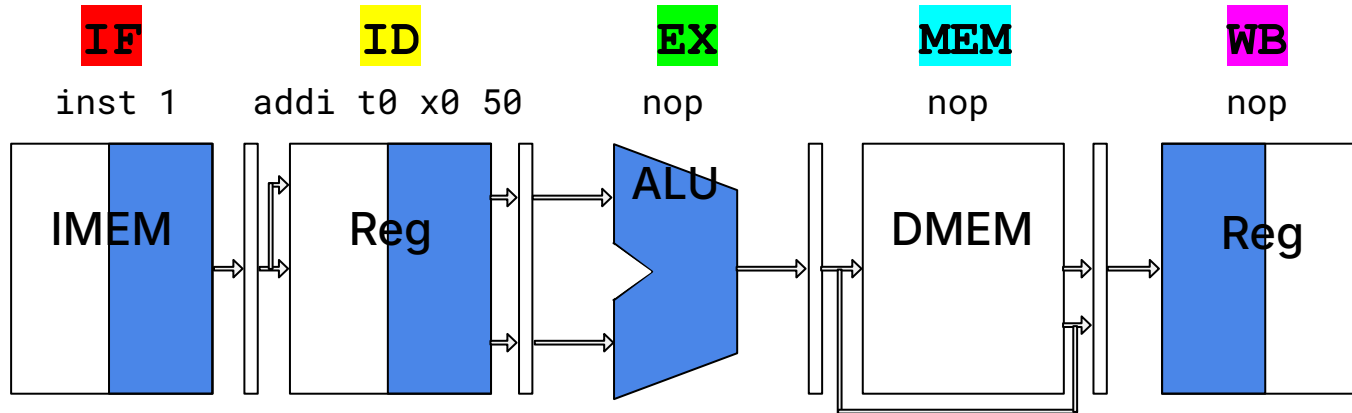
```
sw t2 16(x0)
```

```
nop
```

```
addi t0 x0 11
```

```
nop
```

PC	4
t0	0
t1	0
t2	0x0012F313



Structural Hazard: Cycle 2

addi t0 x0 50

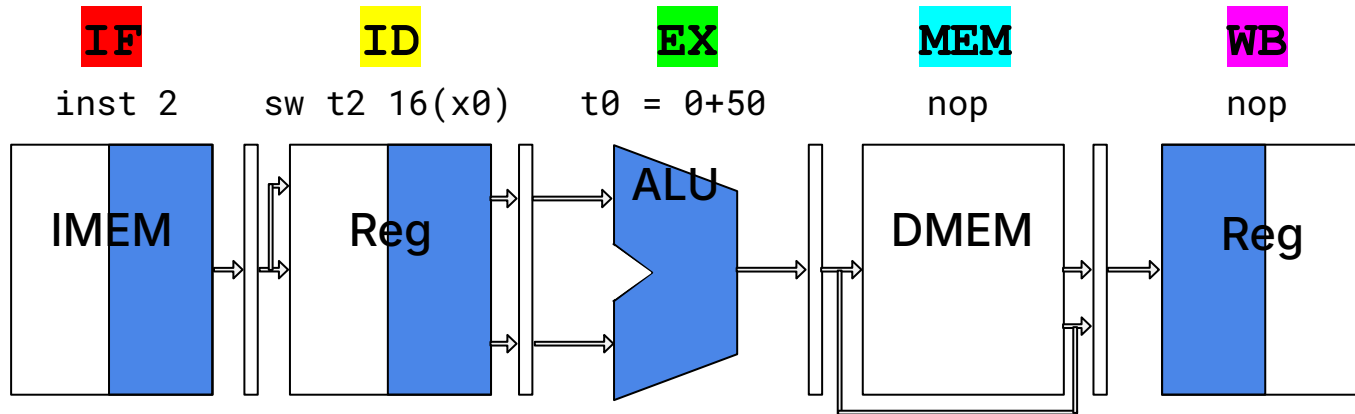
sw t2 16(x0)

nop

addi t0 x0 11

nop

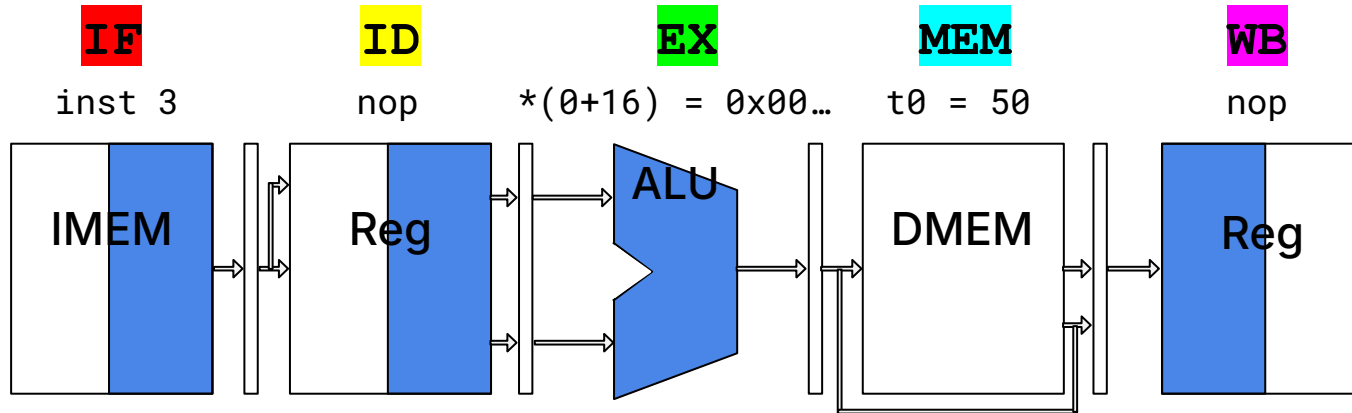
PC	8
t0	0
t1	0
t2	0x0012F313



Structural Hazard: Cycle 3

```
addi t0 x0 50  
sw t1 16(x0)  
nop  
addi t0 x0 11  
nop
```

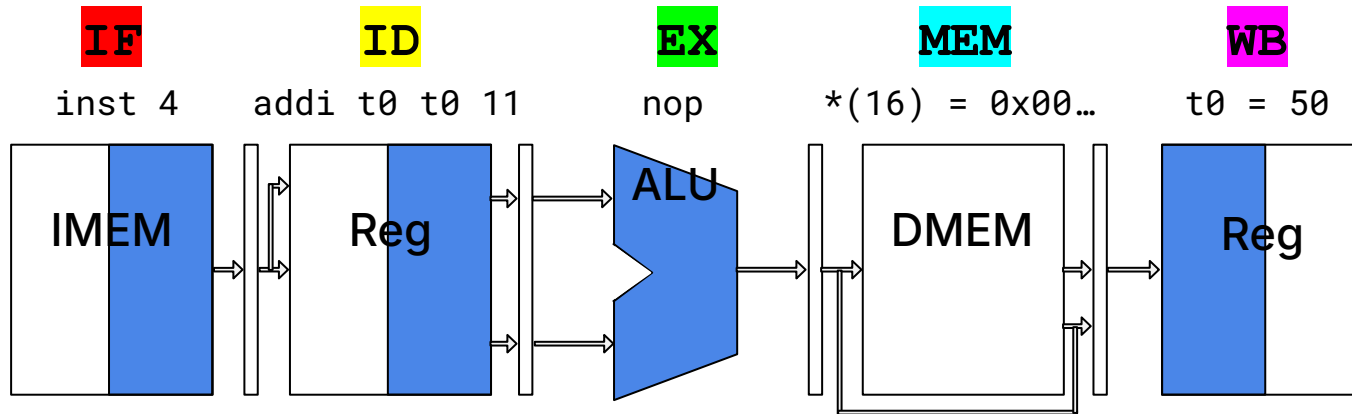
PC	12
t0	0
t1	0
t2	0x0012F313



Structural Hazard: Cycle 4

addi t0 x0 50
sw t1 16(x0)
nop
addi t0 x0 11
nop

PC	16
t0	0
t1	0
t2	0x0012F313

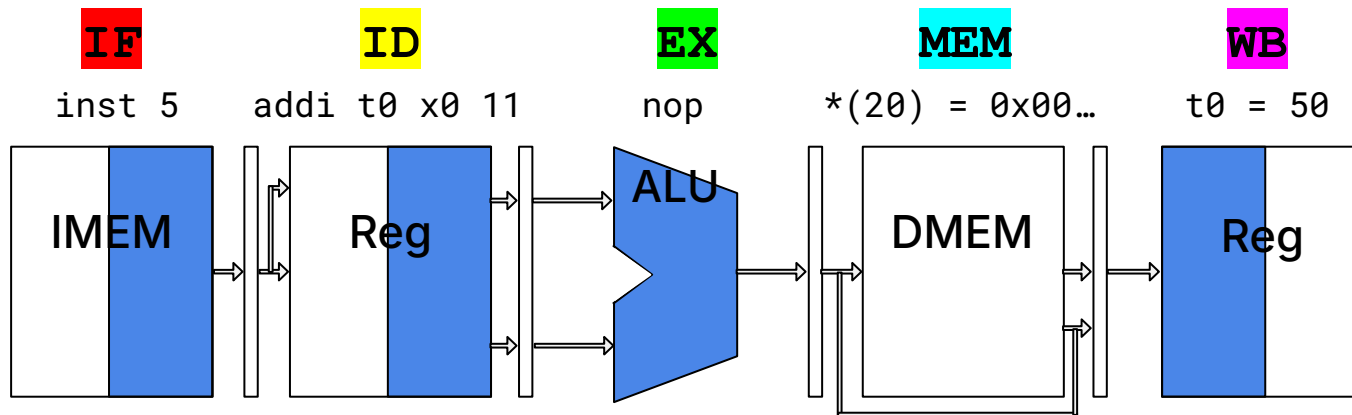


Structural Hazard in RegFile

addi t0 x0 50
sw t1 20(x0)
nop
addi t0 x0 11
nop

Can Regfile read and write at the same time?

PC	16
t0	0
t1	0
t2	0x0012F313

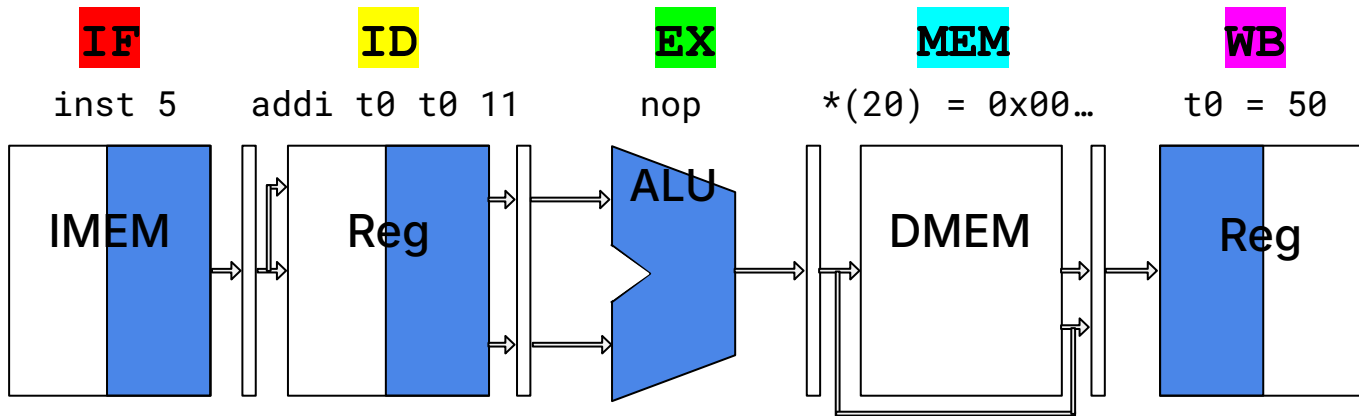


Structural Hazard in Memory

```
addi t0 x0 50  
sw t1 20(x0)  
nop  
addi t0 t0 11  
nop
```

Can memory read and
write to the same location
in the same time?

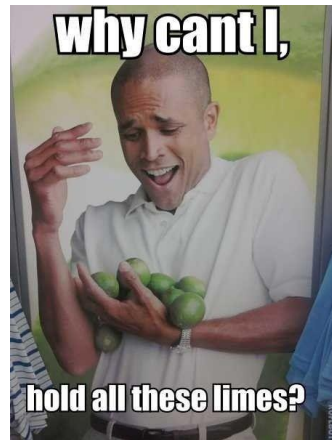
PC	16
t0	0
t1	0
t2	0x0012F313



Structural Hazards

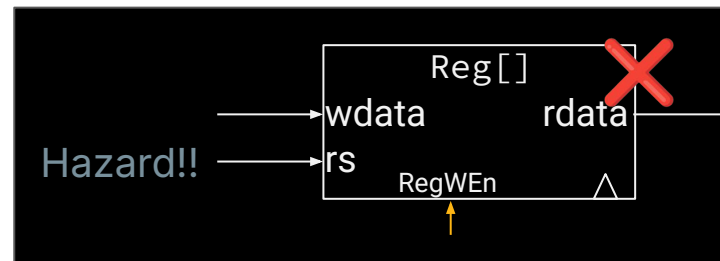
- **Structural hazard:**
 - Hardware does not support access across multiple instructions in the same cycle.
- Occurs when multiple instructions compete for **access to a single physical resource**.
- Two potential structural hazards in RV32I:
 - Simultaneous **RegFile** accesses: `inst4` reads (ID) and `inst1` writes (WB).
 - Simultaneous **memory** accesses: `inst5` (IF) and `inst2` (MEM).
- RV32I solution: Specify hardware requirements!
 - Can always solve structural hazards by adding more HW.

In our current CPU, **structural hazards are not an issue**. We have the needed HW requirements on RegFile and Memory.



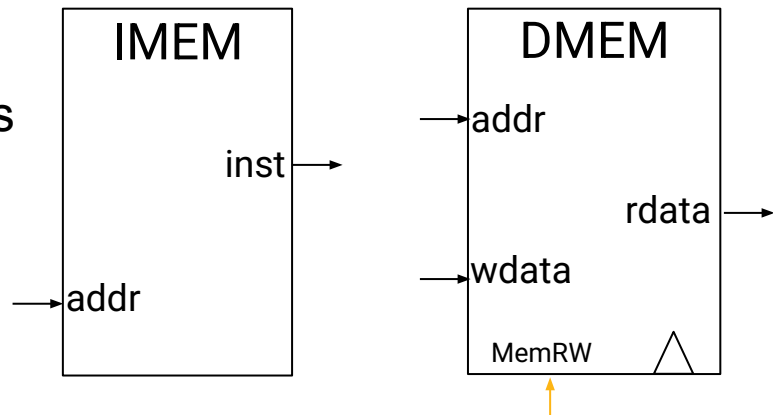
Avoiding Structural Hazards in Regfile

- ✗ A structural hazard could occur if our RegFile hardware did not support simultaneous read/write:
- RV32I's required RegFile design prevents this structural hazard.
 - Independent (separate) ports
 - Three accesses can happen in the same cycle (2 read, 1 write).



Avoiding Structural Hazards in Memory

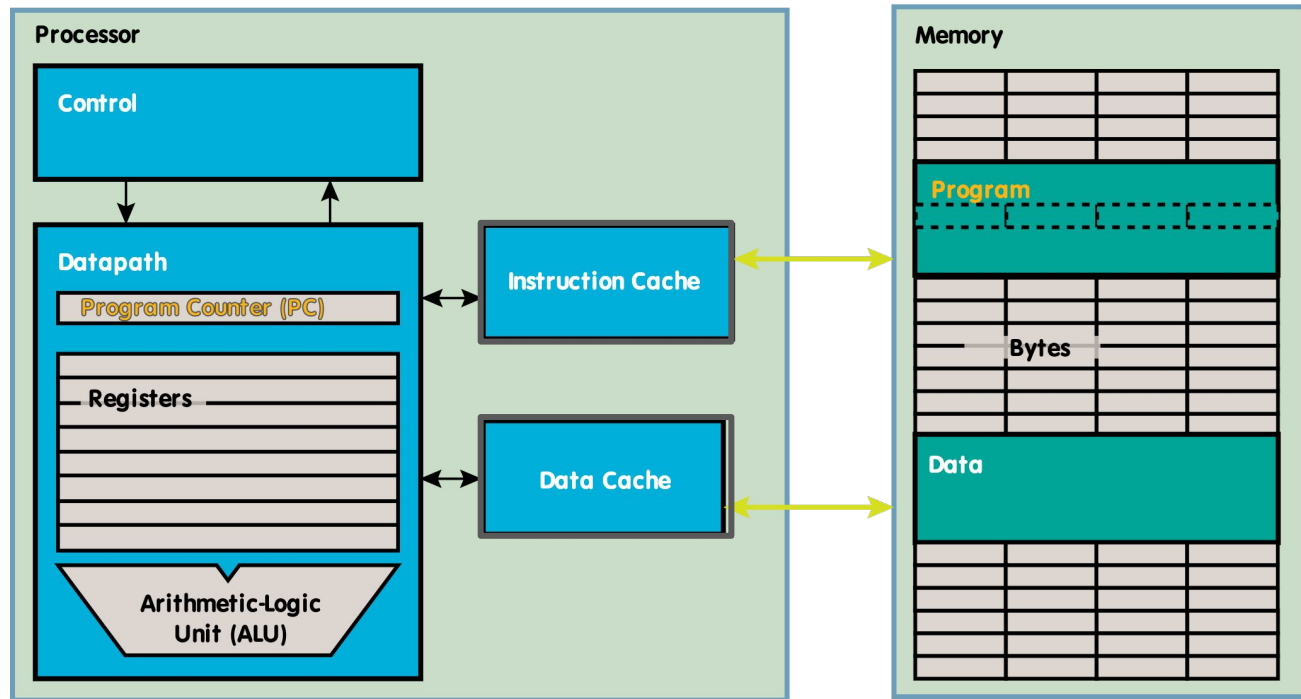
- ✗ A structural hazard could occur if IMEM, DMEM were the same hardware:
- RV32I's required IMEM, DMEM design as separate memories prevents this structural hazard.
 - Remember: IMEM, DMEM are two L1 caches!
- In other words, our CPU actively prevents us from modifying instructions while running code
 - A good idea in general to avoid hackers, but not guaranteed. These protections might not be active on exam questions...



Instruction and Data Caches

- Two fast, separate on-chip memories, one for instructions and one for data:

```
$ # my computer
$ sysctl -a | grep
"hw.l1.*size"
hw.l1icachesize:
131072
hw.l1dcachesize:
65536
$
```



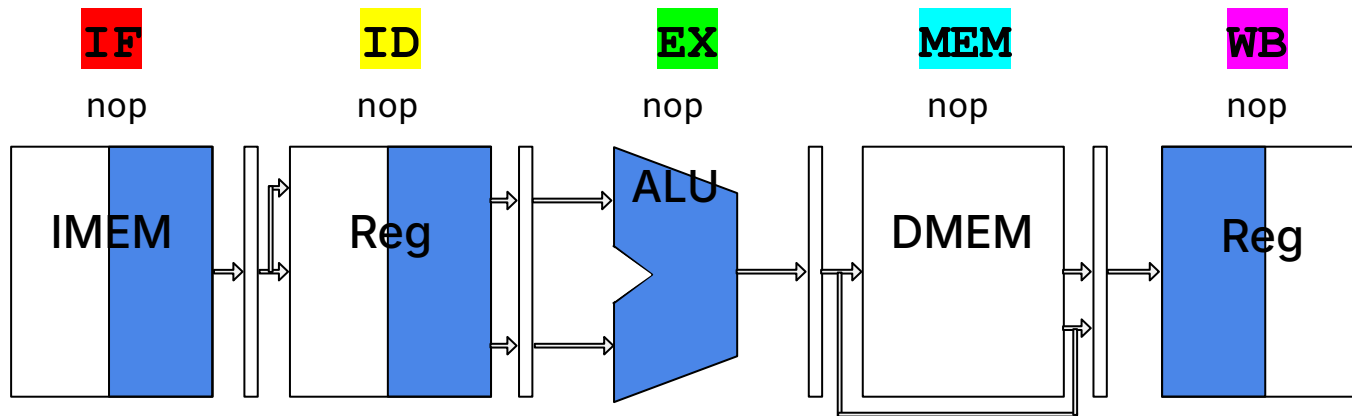
Agenda

- Pipelined RISC-V Datapath
- Hazards Introduction
- Structural Hazards
- **Data Hazards**
 - **What are Data Hazards**
 - Forwarding
 - Loads
- Control Hazards
- Superscalar Processors and Calculating CPI

Data Hazard

```
addi t0 x0 50
addi t1 t0 100
```

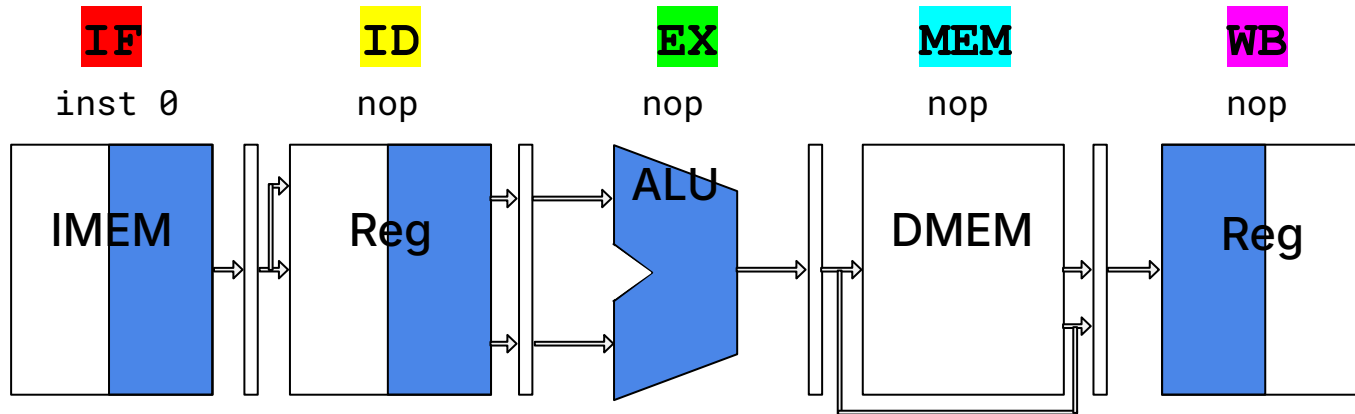
PC	0
t0	0
t1	0
t2	0



Data Hazard: Cycle 0

```
addi t0 x0 50  
addi t1 t0 100
```

PC	0
t0	0
t1	0
t2	0

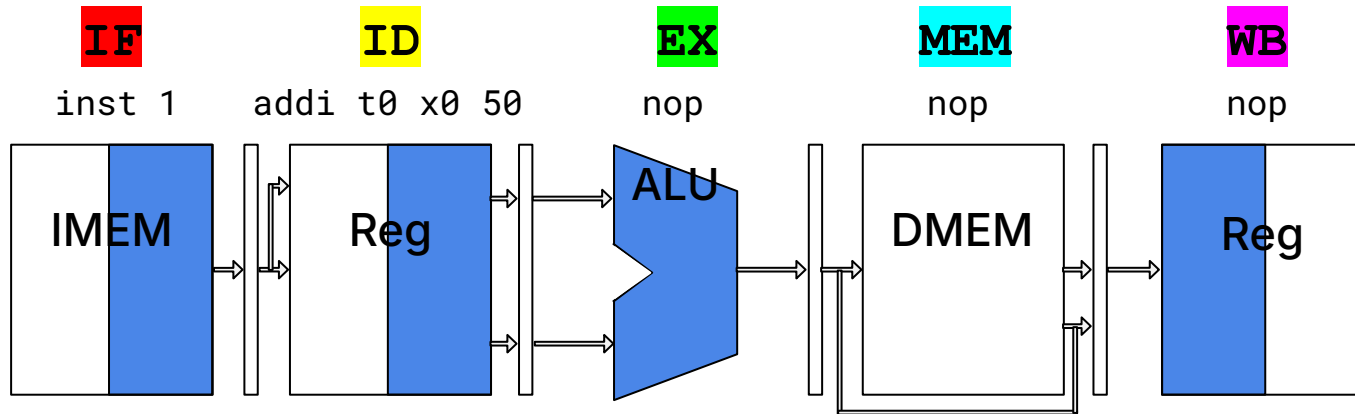


Data Hazard: Cycle 1

```
addi t0 x0 50
```

```
addi t1 t0 100
```

PC	4
t0	0
t1	0
t2	0



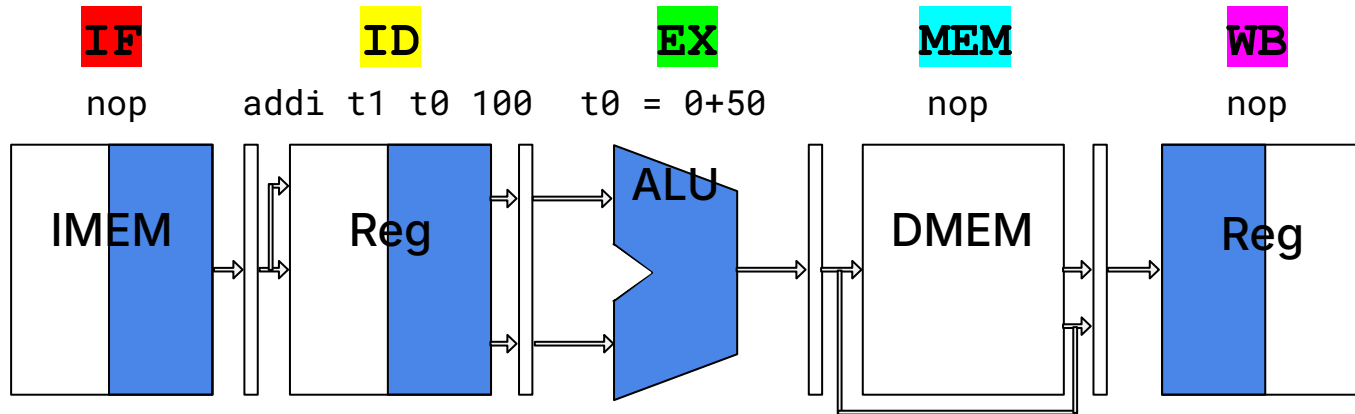
Data Hazard: Cycle 2

```
addi t0 x0 50
```

```
addi t1 t0 100
```

t0 is still 0 when this is ID'd!
How many stalls do we
need to fix this?

PC	8
t0	0
t1	0
t2	0



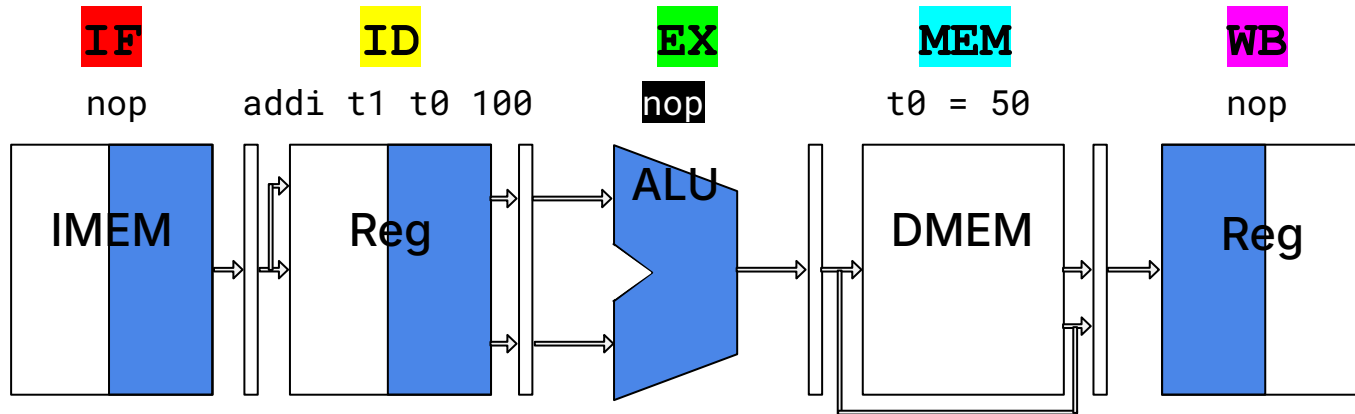
Data Hazard: Cycle 3 (1 stall)

```
addi t0 x0 50
```

```
addi t1 t0 100
```

Still need to stall

PC	8
t0	0
t1	0
t2	0



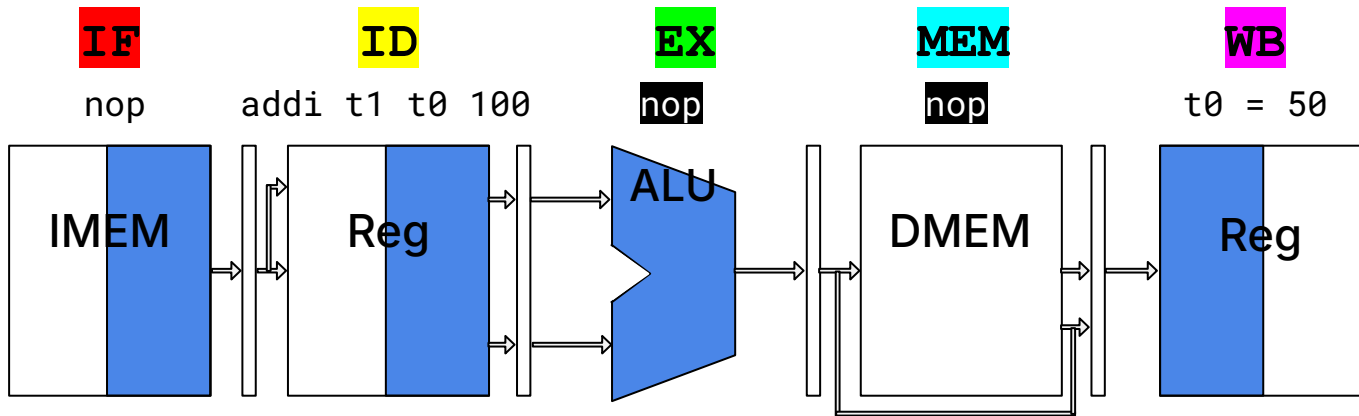
Data Hazard: Cycle 4 (2 stall)

```
addi t0 x0 50
```

```
addi t1 t0 100
```

If register write happens first, no more stalls. If register read happens first, need one more stall

PC	8
t0	0
t1	0
t2	0



Data Hazards

- **Data hazard**
 - Instructions have data dependency.
 - Need to wait for previous instruction to complete its data read/write.
- Depending on instructions, can require up to 2-3 stalls to resolve
- If the same register is read on the same cycle it is written:
 - WB must write value before ID reads new value.
 - **Not structural hazard!** Separate ports allow simultaneous R/W, but there is a data dependency.
- Solution: RegFile HW should **write-then-read** in same cycle.
 - Exploits high speed of RegFile (100 ps + 100 ps)
 - (in some designs, e.g., high frequency, this RegFile design might be impossible)
 - Saves one stall... But we still have two stalls.

Agenda

- Pipelined RISC-V Datapath
- Hazards Introduction
- Structural Hazards
- **Data Hazards**
 - What are Data Hazards
 - **Forwarding**
 - Loads
- Control Hazards
- Superscalar Processors and Calculating CPI

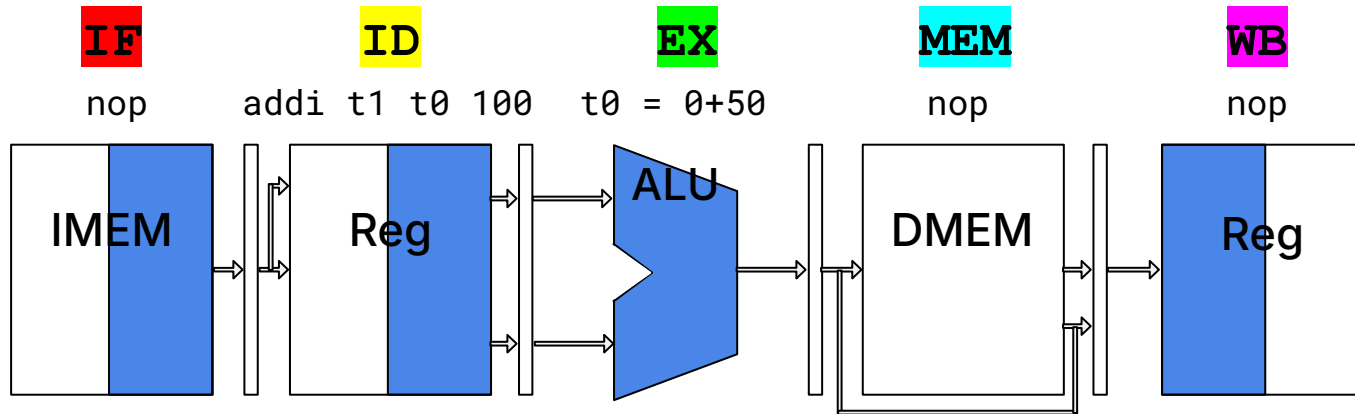
Data Hazard: Cycle 2

```
addi t0, x0, 50
```

```
addi t1, t0, 100
```

t0 is still 0 when this is ID'd!
How many stalls do we need to fix this?

PC	8
t0	0
t1	0
t2	0

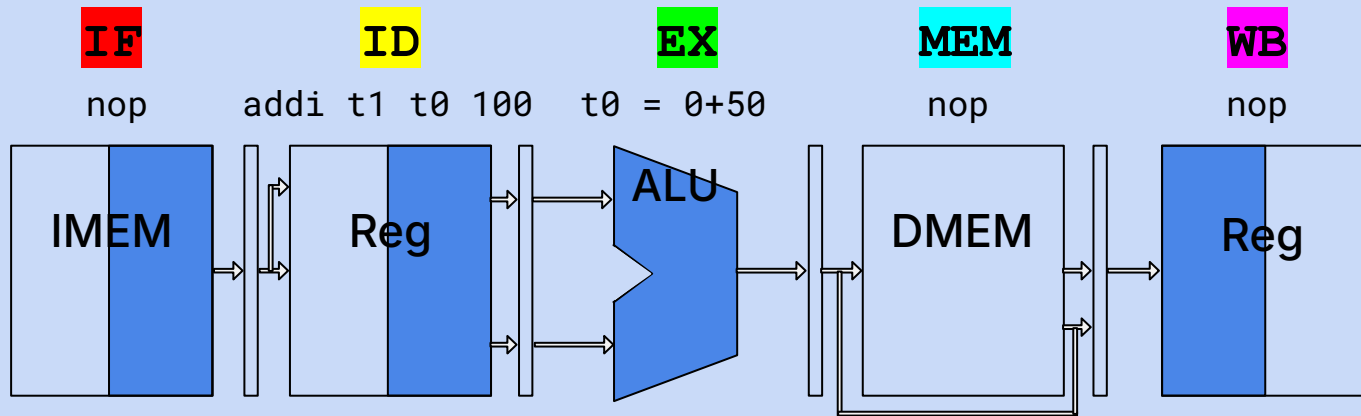


How can we avoid stalling?

```
addi t0 x0 50  
addi t1 t0 100
```

Currently, WB happens too late for ID of next instruction.
Is there a way we can change our pipeline to avoid stalling?

PC	8
t0	0
t1	0
t2	0

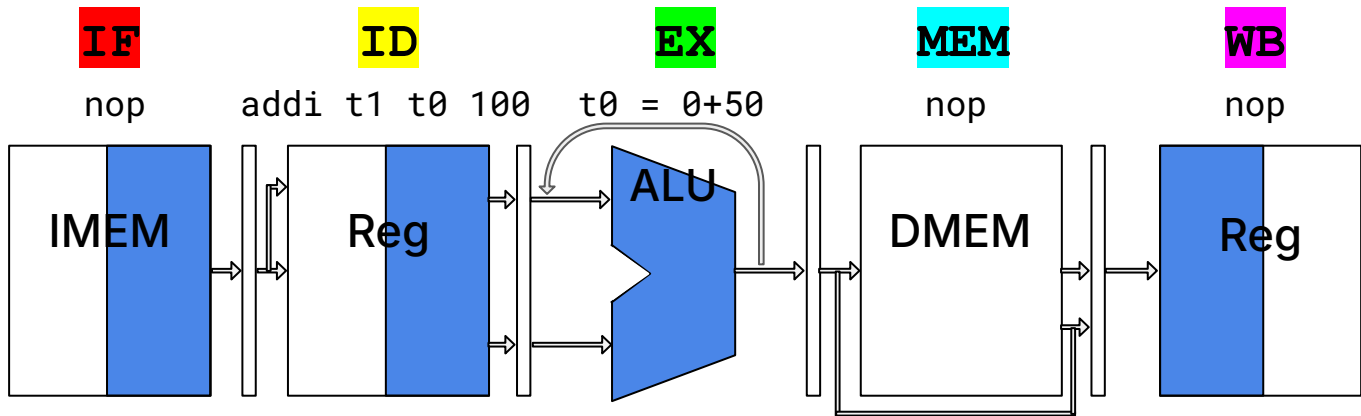


How can we avoid stalling?

```
addi t0 x0 50  
addi t1 t0 100
```

Add a path from ALU output **back** to ALU in, mux with Regfile output. Called a **forwarding path**.

PC	8
t0	0
t1	0
t2	0



Forwarding

Forwarding, aka bypassing, uses the result when it is computed.

- Don't wait for value to be stored into RegFile.
- Instead, grab operand from the pipeline stage.

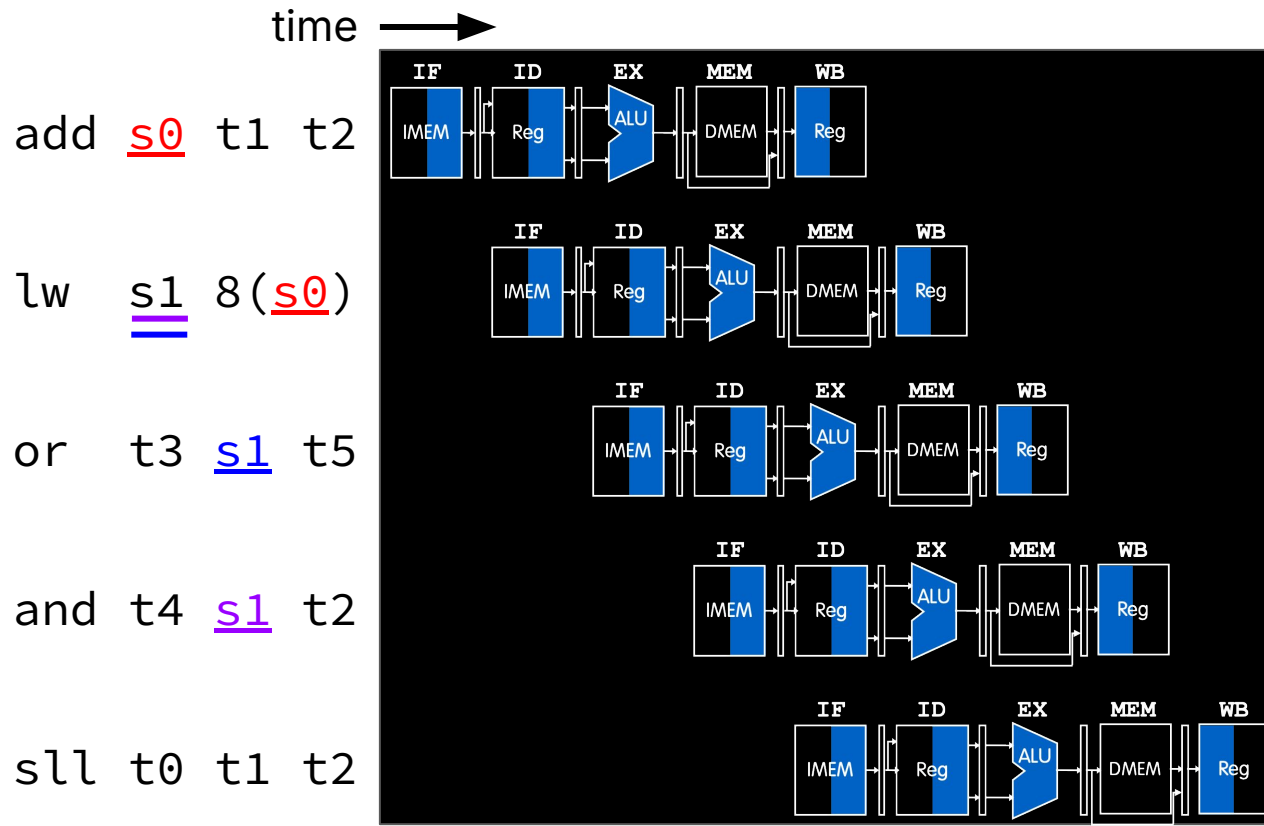
Implementation:

- Make extra connections in the datapath.
 - Also add forwarding control logic.
 - Makes the datapath more complicated, but should resolve most data hazards
- Are there any data hazards that can't be resolved through forwarding?

Agenda

- Pipelined RISC-V Datapath
- Hazards Introduction
- Structural Hazards
- **Data Hazards**
 - What are Data Hazards
 - Forwarding
 - **Loads**
- Control Hazards
- Superscalar Processors and Calculating CPI

(Potential) Pipelined Hazards in RV32I Datapath



Which data hazards can forwarding fix? Select all:

- A. add EX stage output needed by lw MEM input
- B. lw MEM stage output needed by or EX input
- C. lw MEM stage output needed by and EX input
- D. None of the above

Which data hazards can forwarding fix? Select all:

CS 61C

Summer 2026

add EX stage output needed by lw MEM input

lw MEM stage output needed by or EX input

lw MEM stage output needed by and EX input

None of the above



Which data hazards can forwarding fix? Select all:

CS 61C

Summer 2026

- ☐ 0%
- lw MEM stage output needed by or EX input
☐ 0%
- lw MEM stage output needed by and EX input
☐ 0%
- None of the above
☐ 0%



Which data hazards can forwarding fix? Select all:

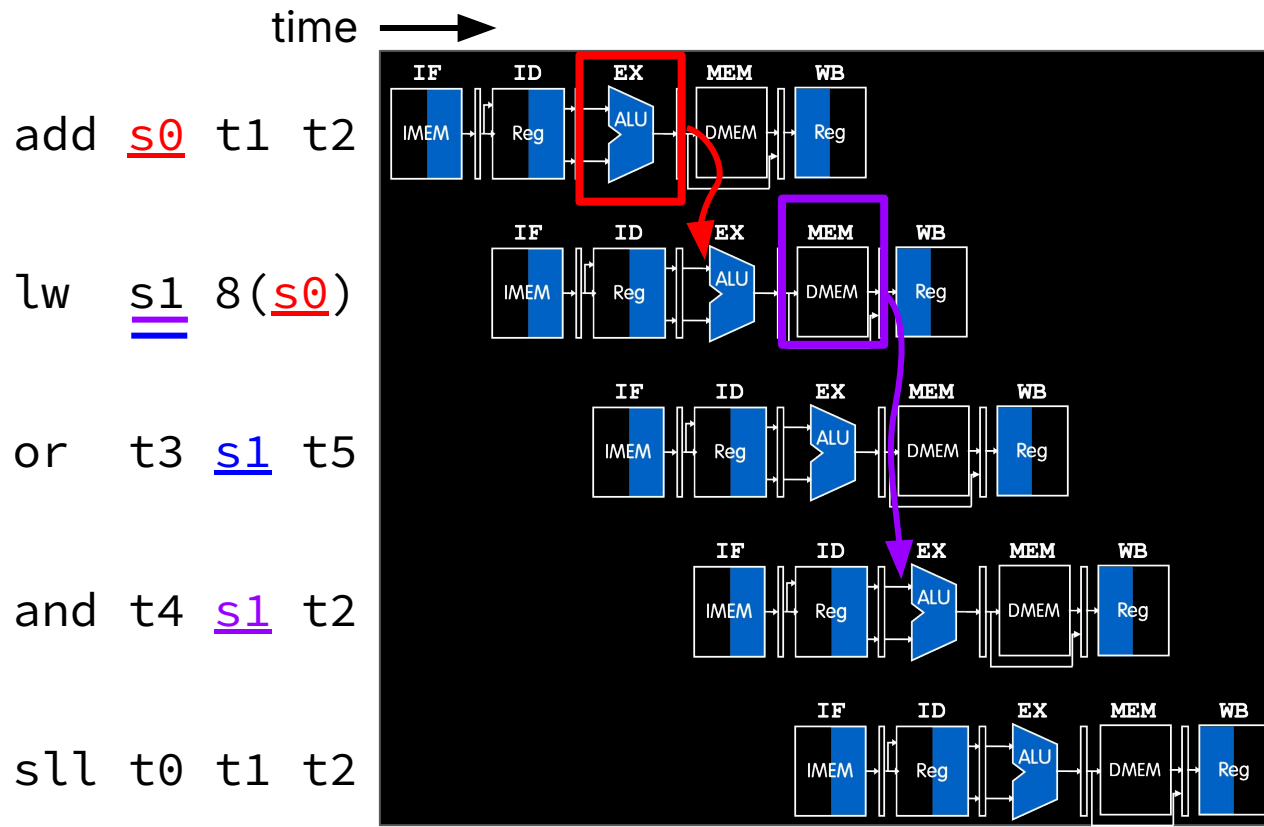
CS 61C

Summer 2026

- ☐ 0%
- lw MEM stage output needed by or EX input
☐ 0%
- lw MEM stage output needed by and EX input
☐ 0%
- None of the above
☐ 0%

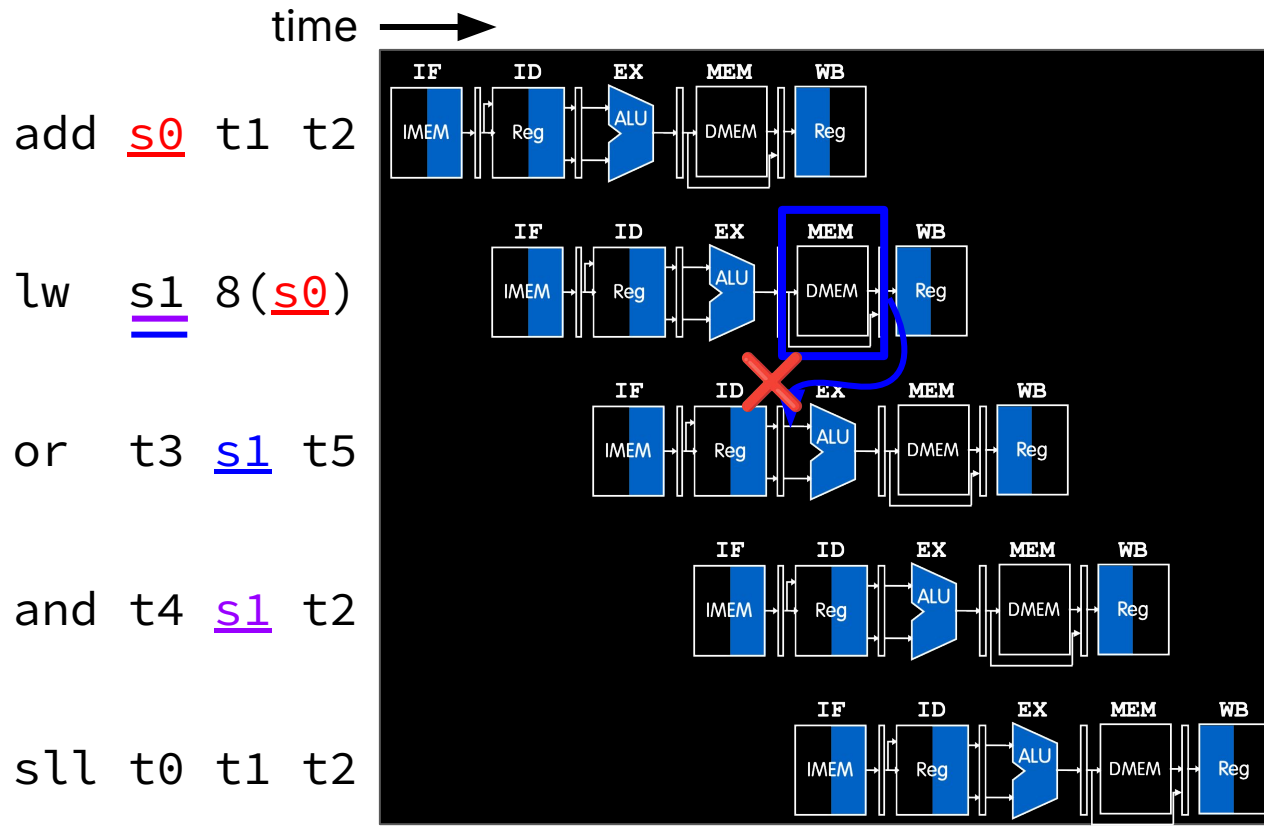


(Potential) Pipelined Hazards in RV32I Datapath



A and C: Data is computed and "forwarded" to the right stage

(Potential) Pipelined Hazards in RV32I Datapath

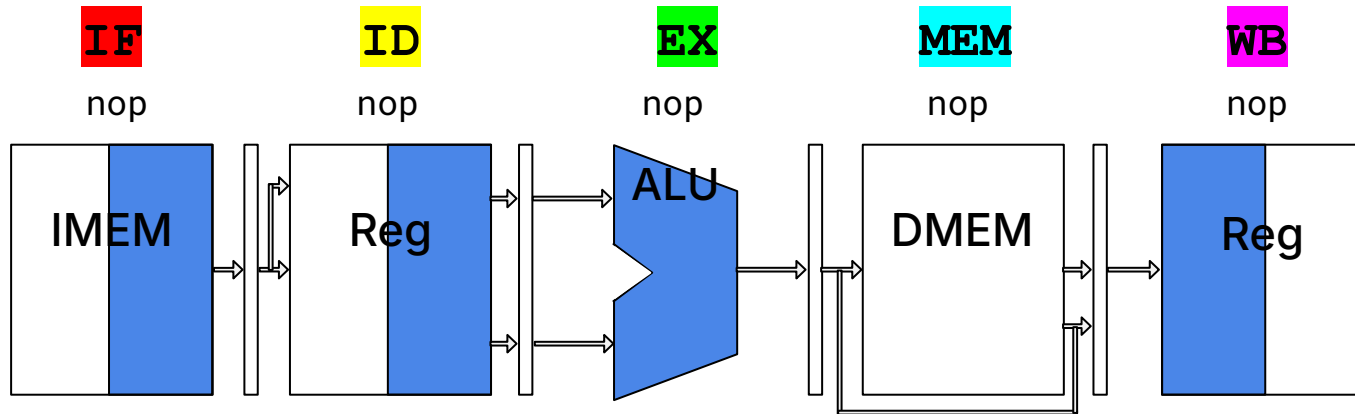


B: Data can't be forwarded, because it would need to go **backward** to arrive in time.

Data Hazard: Load Instructions

```
lw t0 0(sp)
addi t1 t0 100
```

PC	0
t0	0
t1	0
0(sp)	50

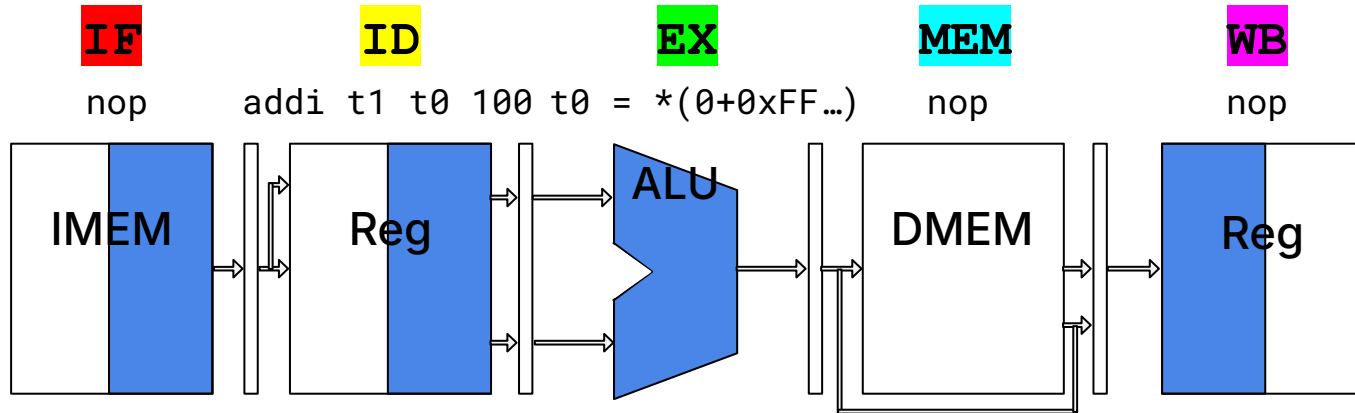


Data Hazard: Load Instructions (Cycle 2)

```
lw t0 0(sp)
```

```
addi t1 t0 100
```

PC	8
t0	0
t1	0
0(sp)	50



Data Hazard: Load Instructions (Cycle 2)

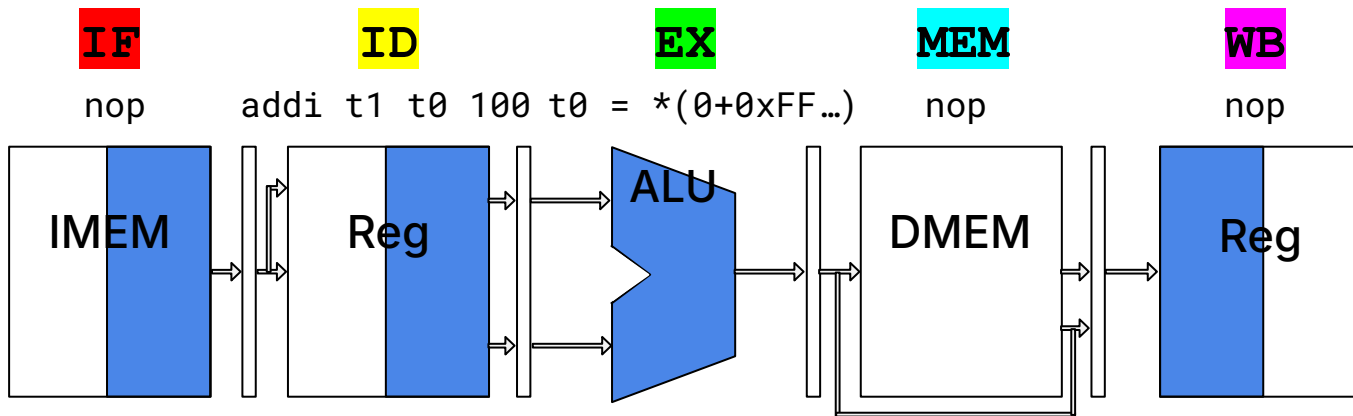
```
lw t0 0(sp)
```

```
addi t1 t0 100
```



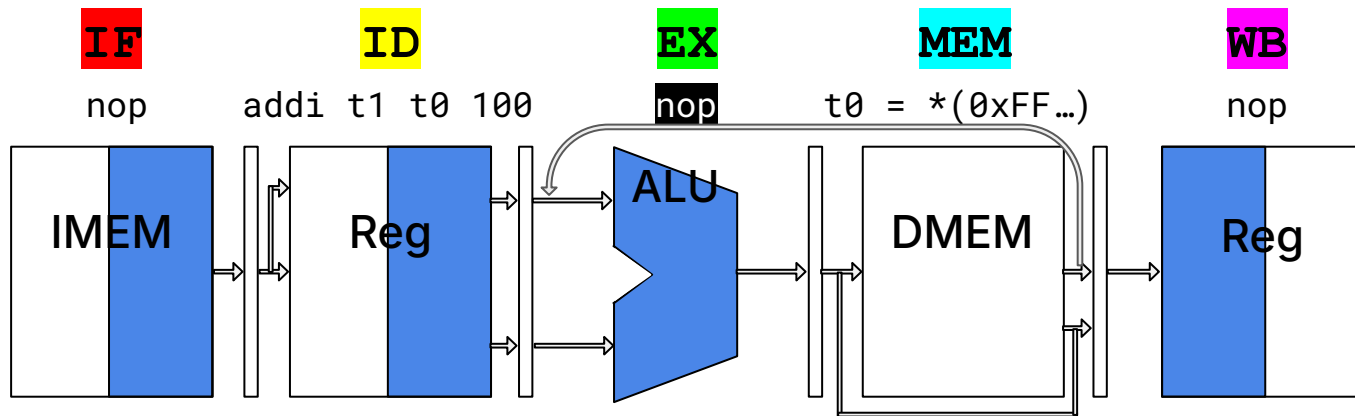
We haven't figured out t0 at this time! The forwarding path would need to time travel.

PC	8
t0	0
t1	0
0(sp)	50



Data Hazard on Loads

- The instruction slot after a load is called the **load delay slot**.
- If this instruction uses the result of load:
 - The hardware must **stall for one cycle** (plus **forwarding**).
 - This results in performance loss!
- No way to prevent this at the hardware level. But...



Code Scheduling

A naive compiler might create this assembly given the below C code. Takes 9 cycles with two stalls

C code

```
A[3] = A[0] + A[1];  
A[4] = A[0] + A[2];
```

&a		a0
A[0]		t0
A[1]		t1
A[2]		t2
A[3]		t3
A[4]		t4

Simple compilation

(9 cycles for 7 instr.)

```
lw    t0    0(a0)  
lw    t1    4(a0)  ↘ stall+forward:  
add    t2    t0 t1    (+1 cycle)  
sw    t2    12(a0)  
lw    t3    8(a0)  ↘ (+1 cycle)  
add    t4    t0 t3  
sw    t4    16(a0)
```

Code Scheduling

A clever compiler will **reorder** instructions to put an unrelated instruction in the load delay slot. This effectively saves that stall. No performance loss!

Code scheduling:

With knowledge of the underlying CPU pipeline, the compiler reorders code to improve performance.

Simple compilation
(**9 cycles** for 7 instr.)

```
lw  t0  0(a0)
lw  t1  4(a0)
add  t2  t0 t1
sw   t2  12(a0)
lw   t3  8(a0)
add  t4  t0 t3
sw   t4  16(a0)
```



✓ Alternative
(7 cycles):

```
lw  t0  0(a0)
lw  t1  4(a0)
lw  t3  8(a0)
add  t2  t0 t1
sw   t2  12(a0)
add  t4  t0 t3
sw   t4  16(a0)
```

forward (+0 cycle)

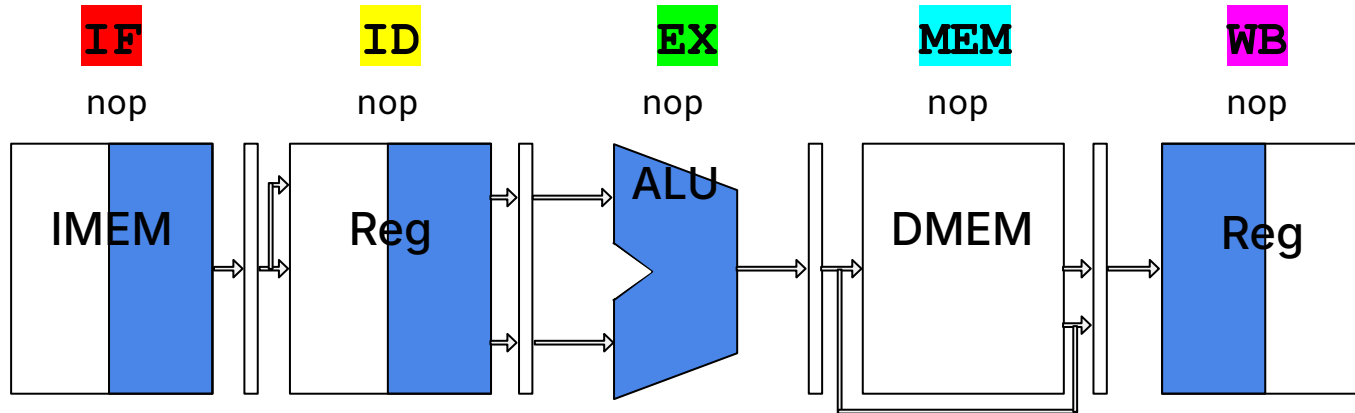
Agenda

- Pipelined RISC-V Datapath
- Hazards Introduction
- Structural Hazards
- Data Hazards
 - What are Data Hazards
 - Forwarding
 - Loads
- **Control Hazards**
- Superscalar Processors and Calculating CPI

Control Hazard

```
jal x0 8  
addi t0 x0 50  
addi t1 x0 50
```

PC	0
t0	0
t1	0
t2	0



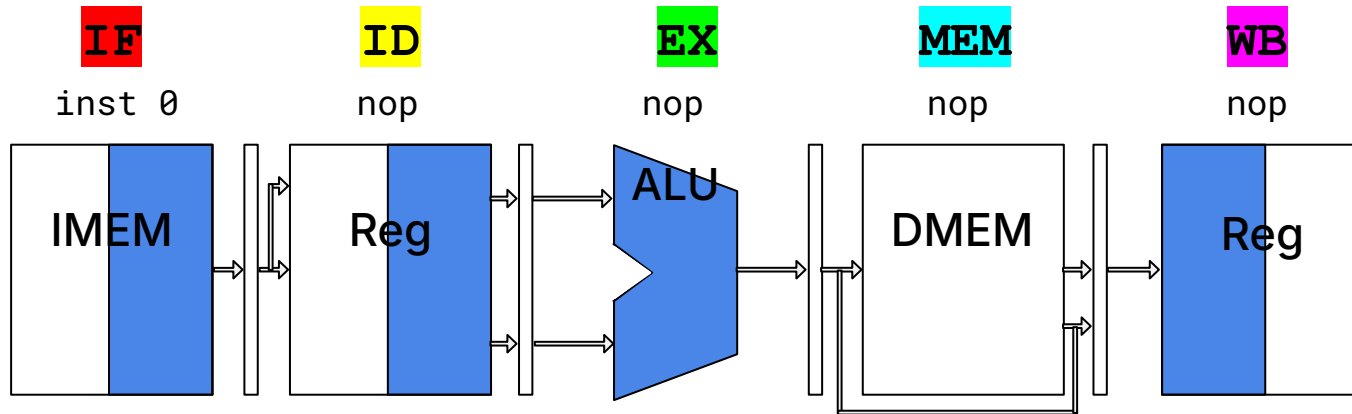
Control Hazard: Cycle 0

```
jal x0 8
```

```
addi t0 x0 50
```

```
addi t1 x0 50
```

PC	0
t0	0
t1	0
t2	0



Control Hazard: Cycle 1

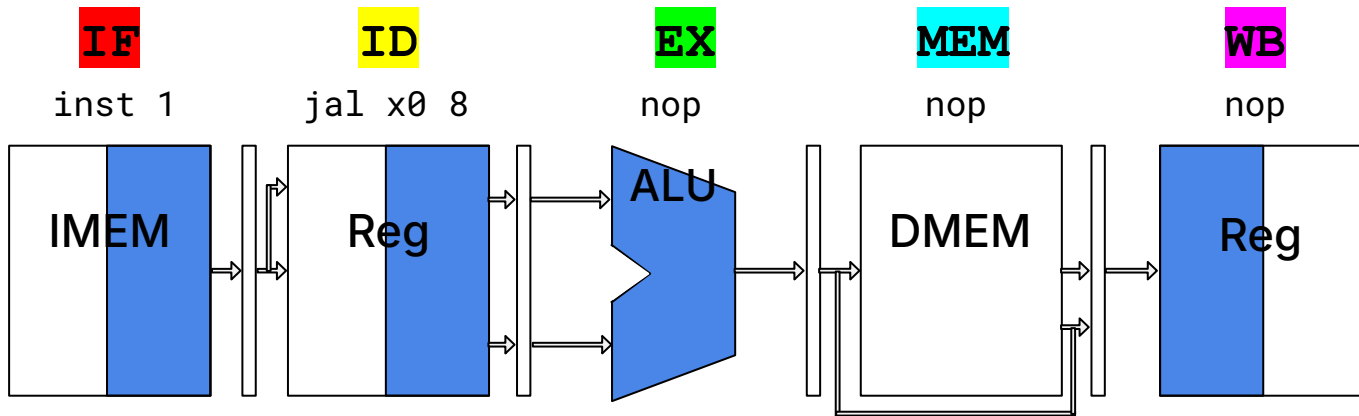
jal x0 8

addi t0 x0 50

addi t1 x0 50

We haven't figured out we have a jal yet... So we load instruction 1 instead of the right PC...

PC	4
t0	0
t1	0
t2	0



Control Hazard: Cycle 1

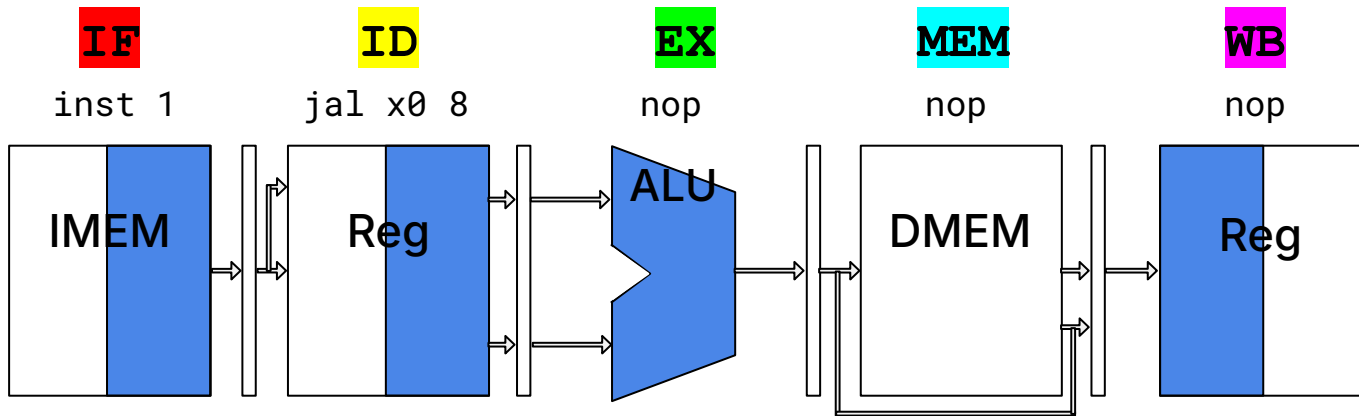
jal x0 8

addi t0 x0 50

addi t1 x0 50

We haven't figured out we have a jal yet... So we load instruction 1 instead of the right PC...

PC	4
t0	0
t1	0
t2	0



Control Hazard: Cycle 2

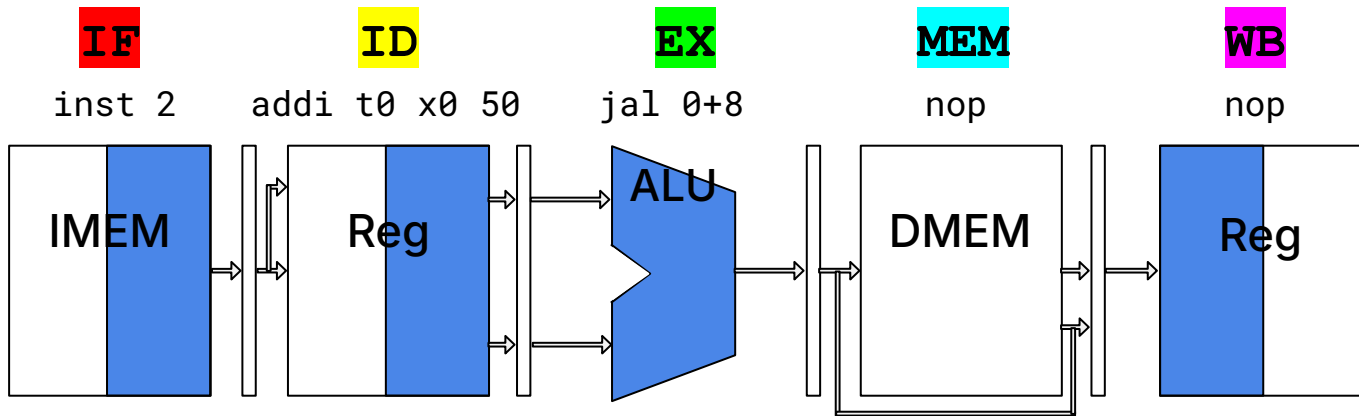
jal x0 8

addi t0 x0 50

addi t1 x0 50

At the end of jal's ID, we can "realize" we have a jal. Need to fix two things: IF and ID (datapath never reaches this state, see fixes in next two slides)

PC	8
t0	0
t1	0
t2	0



Control Hazard: Cycle 2

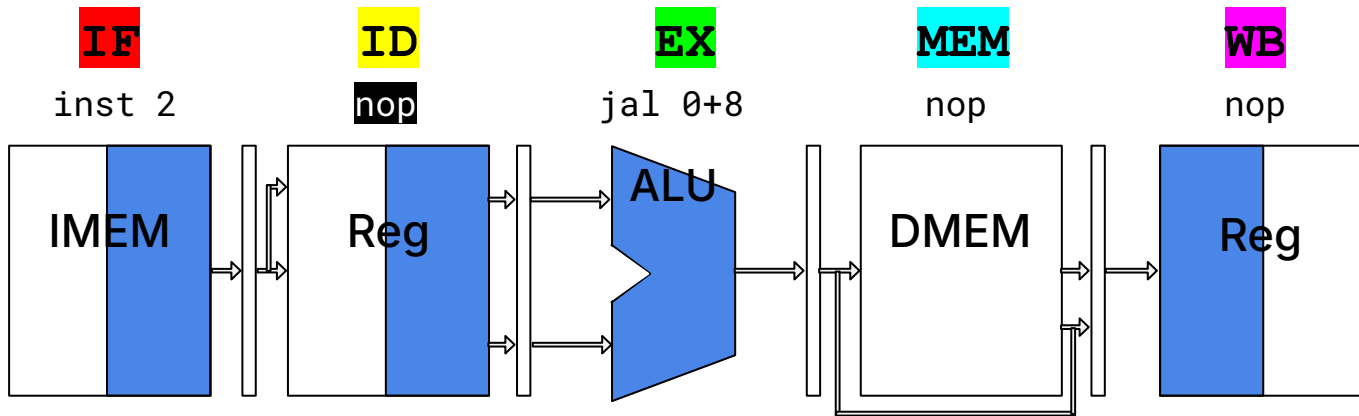
jal x0 8

addi t0 x0 50

addi t1 x0 50

ID fix: Replace the existing instruction with nop. Called **flushing** the pipeline

PC	8
t0	0
t1	0
t2	0



Control Hazard: Cycle 2

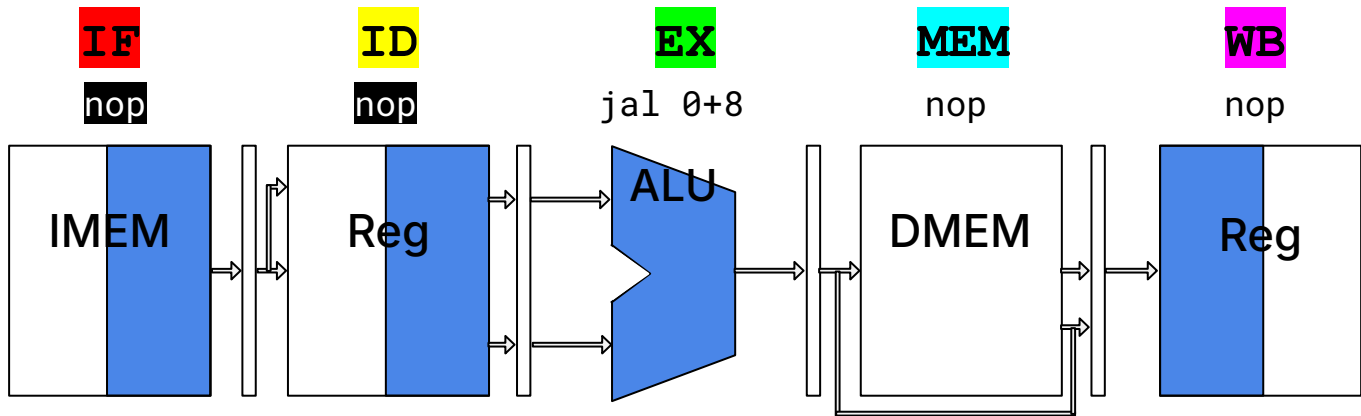
```
jal x0 8
```

```
addi t0 x0 50
```

```
addi t1 x0 50
```

IF fix: Insert nops until we learn where to jump

PC	???
t0	0
t1	0
t2	0



Control Hazard: Cycle 3

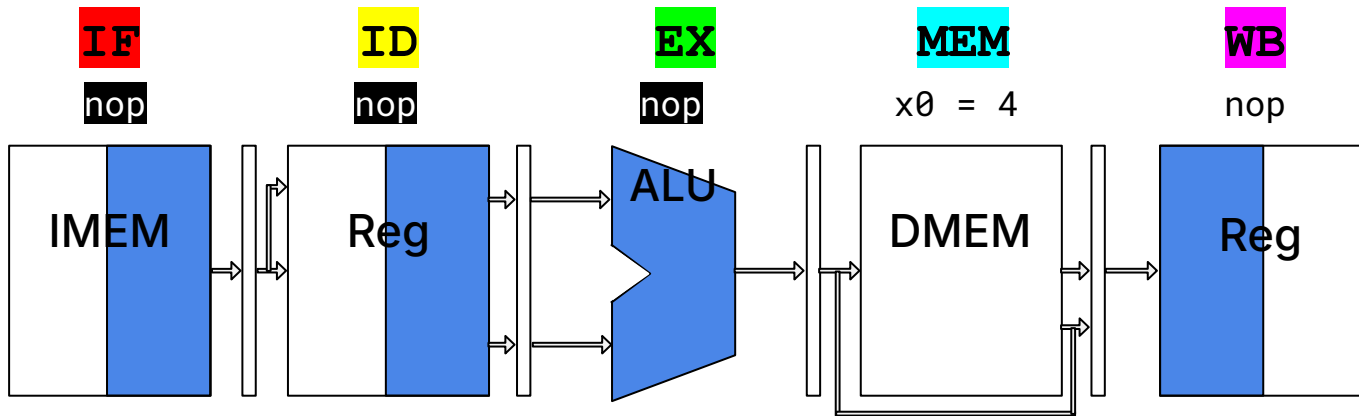
jal x0 8

addi t0 x0 50

addi t1 x0 50

Once EX runs, we can forward new PC to IF to get next instruction. **jal** keeps going to set rd

PC	???
t0	0
t1	0
t2	0



Control Hazard: Cycle 4

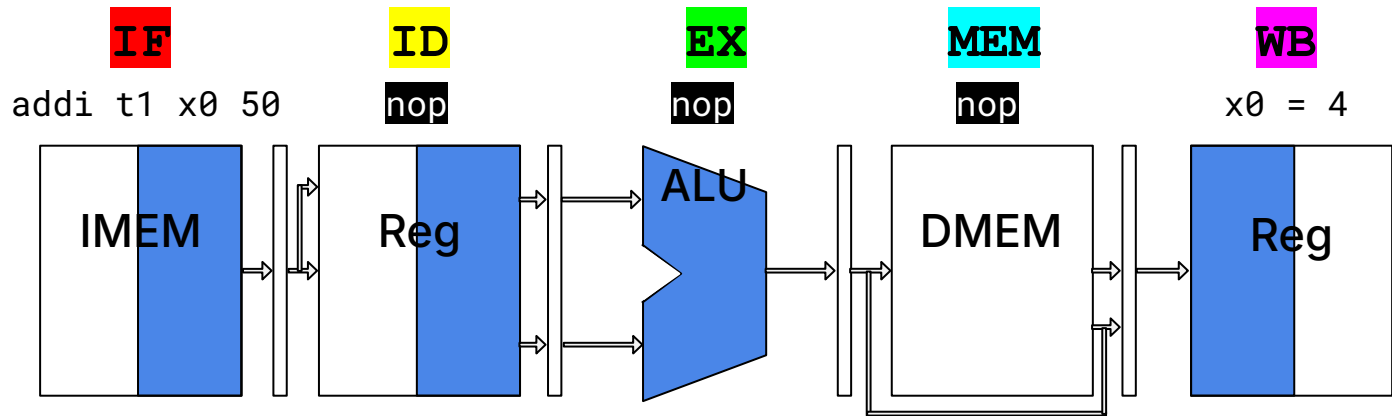
jal x0 8

addi t0 x0 50

addi t1 x0 50

Once EX runs, we can forward new PC to IF to get next instruction. jal keeps going to set rd

PC	8
t0	0
t1	0
t2	0



Control Hazards

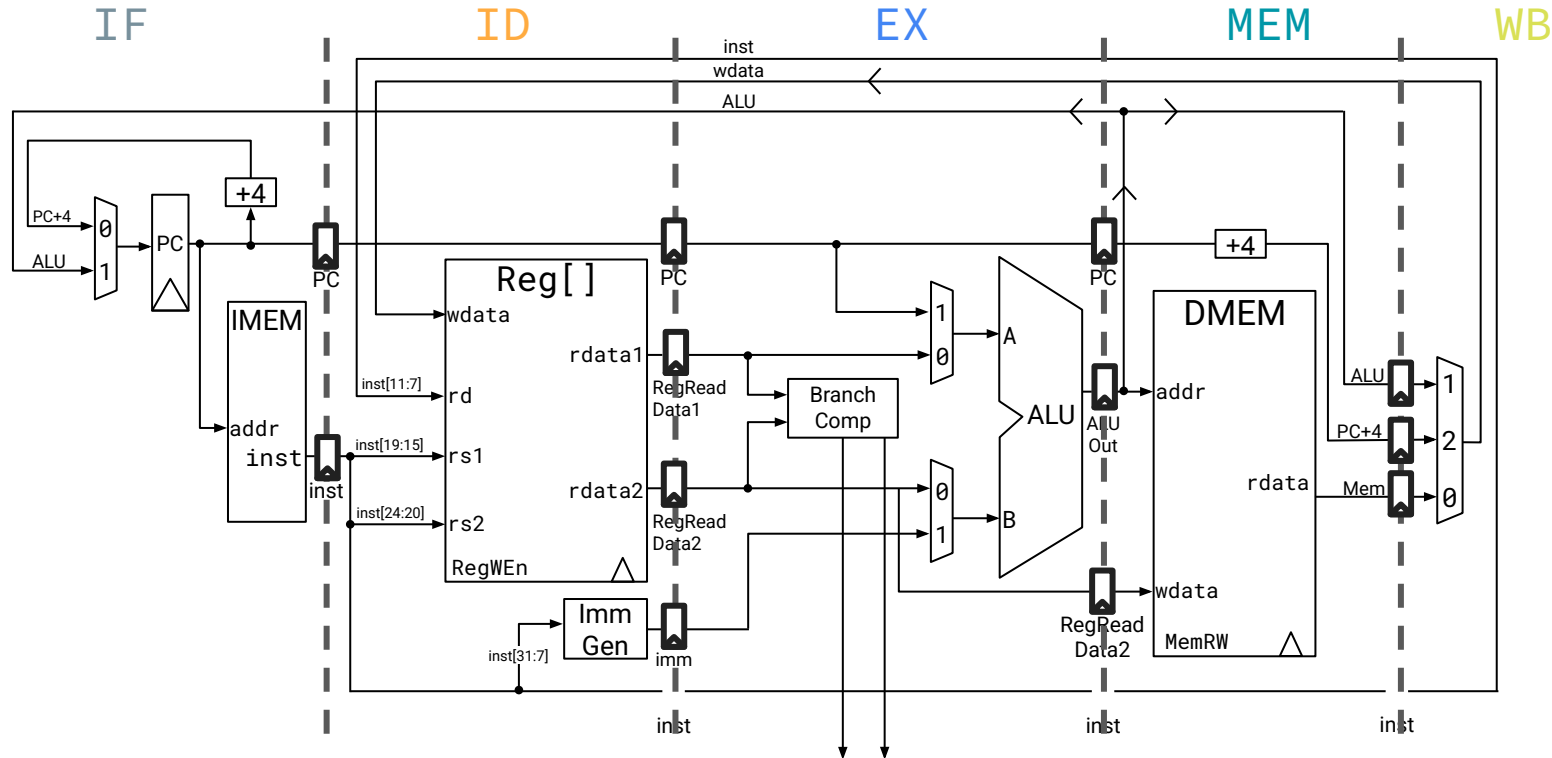
- **Control hazard**

- Flow of execution depends on previous instruction.

Control hazards occur when the instruction fetched may not be the one needed.

- For jump instructions, takes three stalls to resolve (because wire from ALU to PC Mux comes after the EX-MEM pipeline registers)
- For branches and JALR, also will naively take three stalls to resolve (branch comparator in EX, but still needs to wait for ALU value)
 - Once we notice we have a branch/jump, we flush the pipeline and wait for the instruction to tell us next address to jump to)

Pipelined CPU: ALU out to PC received during MEM

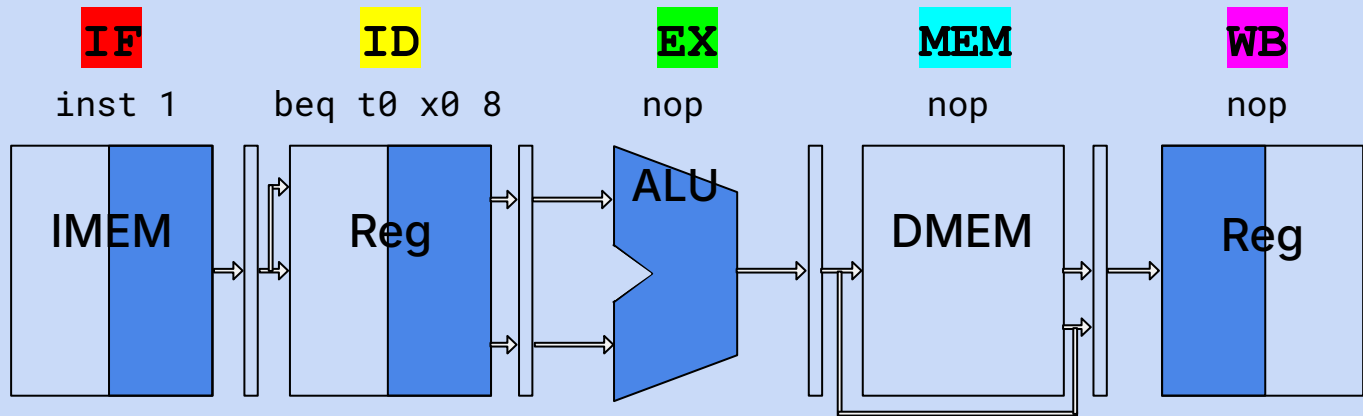


How to fix Control Hazards?

```
beq t0 x0 8  
addi t0 x0 50  
addi t1 x0 50
```

How can we fix control Hazards?

PC	4
t0	0
t1	0
t2	0



Ways to Resolve Control Hazards

- Find a way to compute jump amount earlier?
 - PC and imm are known by ID, and Immgen is faster than Regfile; could maybe use a separate adder to forward jump location sooner
 - Could set up a cache to remember where jumps go to (except for JALRs) - requires a lot of extra circuitry, but can determine new PC just from current PC.
 - Out of scope for this class, but without one of these, the next optimization can't be generalized.
- Run following instructions on branches even if they're the wrong ones.
 - If a branch ends up not being taken, don't flush the instructions, so there's no penalty
 - If branch is taken, flush the pipeline before it affects any state elements
- The above is a simple form of **branch prediction**
 - Predict which way a branch will go based on past history or metadata from compiler, then load appropriate instructions, flushing if we predicted wrong
 - In our current CPU, predicting that we take a branch doesn't help (since we stall three times anyway), but datapaths with more pipeline stages/cached jump locations can make this viable.

Branch Prediction to Reduce Penalties

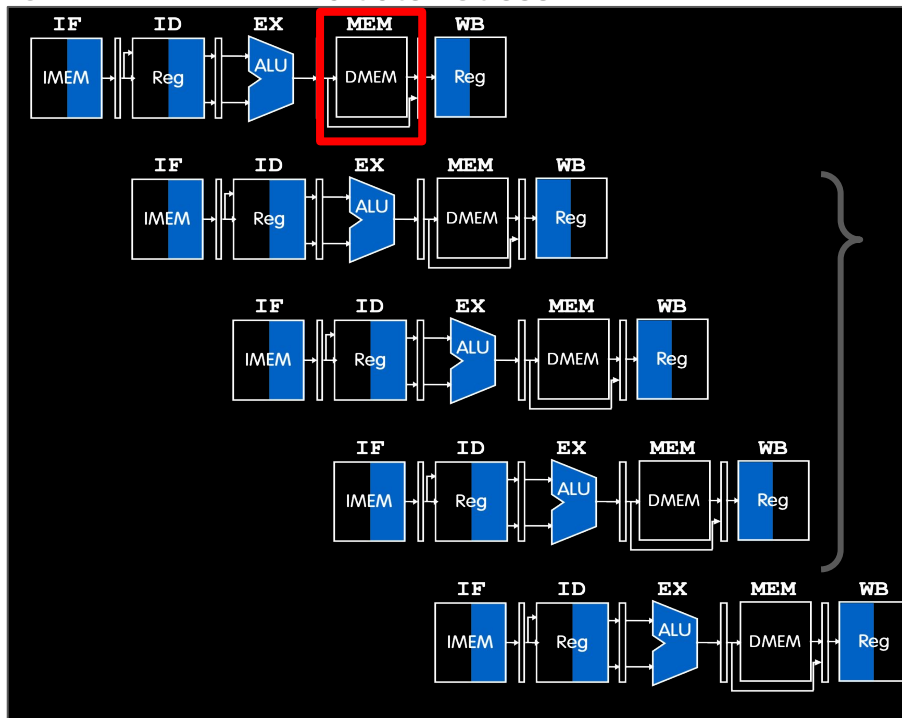
- Every taken branch in the simple RV32I pipeline costs **3 clock cycles**.
 - Branch results are computed during MEM.
 - (Note: if branch is not taken, then pipeline is not stalled; the correct instructions are correctly fetched sequentially after the branch instruction.)
 - (See textbook for an **out of scope** hardware cost improvement.)
- **Branch prediction** can improve the average CPU performance.
 - Early in the pipeline, guess which way branches will go.
 - Flush pipeline if branch prediction was incorrect.

Naïve Predictor Interpretation: Don't Take Branch

"Guess" PC to be PC + 4

"Evaluate" Guess

```
0x40  
beq  t0 t1 Label  
0x44  
sub  t2 s0 t0  
0x48  
or   t6 a0 t0  
0x4c  
xor  t5 t1 s0  
0x50  
add  t2 s0 s0
```



The simple RV32I pipeline effectively always “predicts” that branches are **not taken**.

If branch not taken, correct instructions are already executed.

Penalty for incorrect prediction is still 3 clock cycles! Branch prediction tries to improve **average performance**.

Agenda

- Pipelined RISC-V Datapath
- Hazards Introduction
- Structural Hazards
- Data Hazards
 - What are Data Hazards
 - Forwarding
 - Loads
- Control Hazards
- **Superscalar Processors and Calculating CPI**

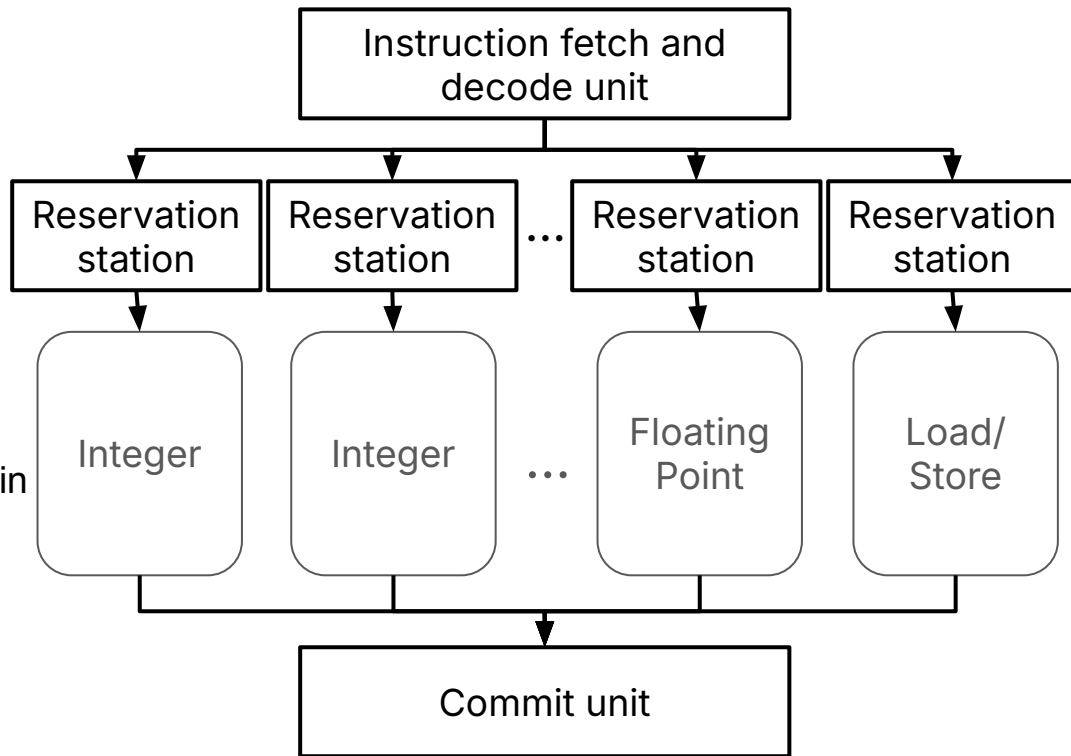
Further Increasing Processor Performance?

1. Increase clock rate.
 - Limited by technology and power dissipation
2. Increase pipeline depth.
 - “Overlap” instruction execution through deeper pipeline, e.g., 10 or 15 stages.
 - Less work per stage □ shorter clock cycle/lower power
 - But more potential for all three types of hazards! (more stalling □ $CPI > 1$)
3. Design a **superscalar processor**.
 - Desktops, laptops, cell phones, etc. often have a few of these, combined with simpler 5-stage pipeline processors.

Superscalar Processors

CPI < 1: multiple instructions completed per clock cycle (IPC > 1)

- Multiple-issue: Start **multiple instructions** per clock cycle.
 - Multiple execution units execute instructions in parallel.
 - Each execution unit has its own pipeline.
- Dynamic “out-of-order” execution:
 - Reorder instructions dynamically in HW to reduce impact of hazards
- Contrast with multicore (separate threads per core)



Superscalar Processors: CPI

$$\frac{\text{Time}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \frac{\text{Cycles}}{\text{Instruction}} \frac{\text{Time}}{\text{Cycle}}$$

- For **superscalar processors**, $\text{CPI} < 1$ because **IPC** > 1 (multiple instructions completed per clock cycle).
- Measure CPI on various benchmarks:
 - Known benchmarks from a variety of application domains:
 - Data compression, code compilation, video decoding, network simulation, etc.
- ARM Cortex-A53 Core: 2 GHz, **dual-issue** processor:
 - 4 BIPS (billion instructions per second), **Peak CPI = 0.5**.
 - In practice, instruction/pipeline dependencies reduce performance.

ARM A53 Benchmark

CS 61C

Summer 2026

